

CSC207 Final Project Presentation Fall 2025

Group Number TUT0401-BA3195-02

Movie Tracker App – User Stories

Find & Organize Movies

Search Movies

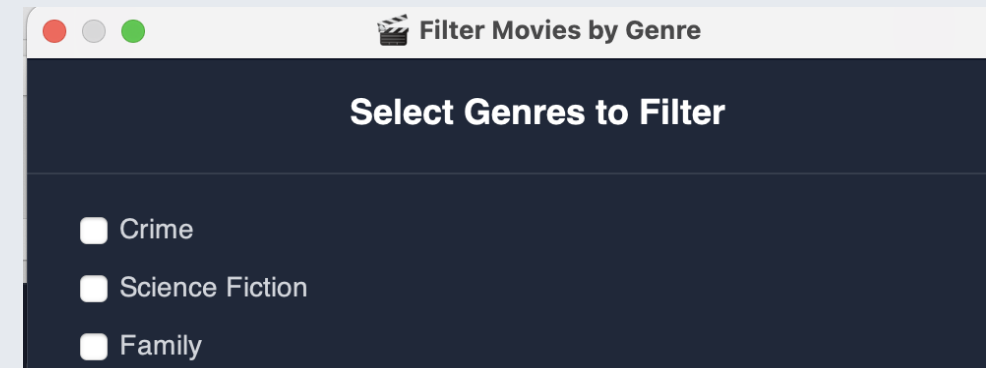
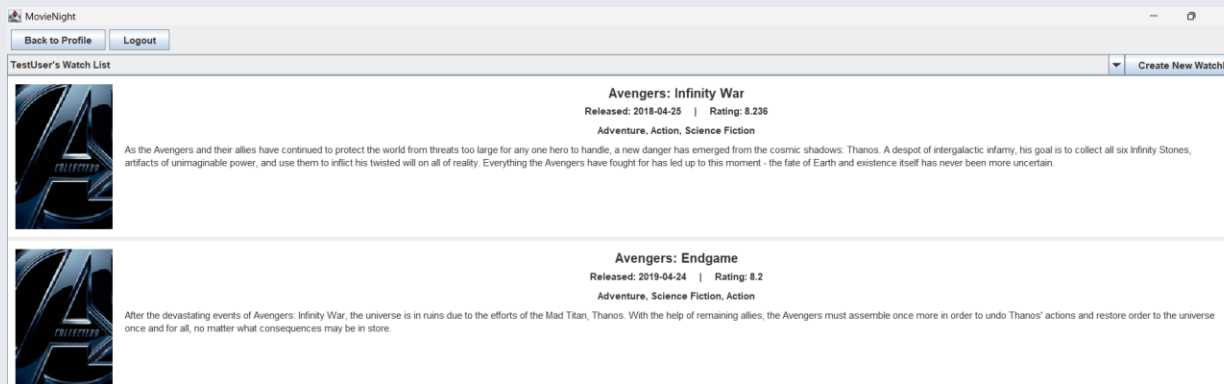
Type any movie name → Get all the relevant results with movie details

Filter by Genre

Ex. choose "Action" → See all action movies

Create & View Watchlists

Save movies → Organize them into 1+ watchlists



Movie Tracker App – User Stories

Review & View Movie Record

★ Rate & Review

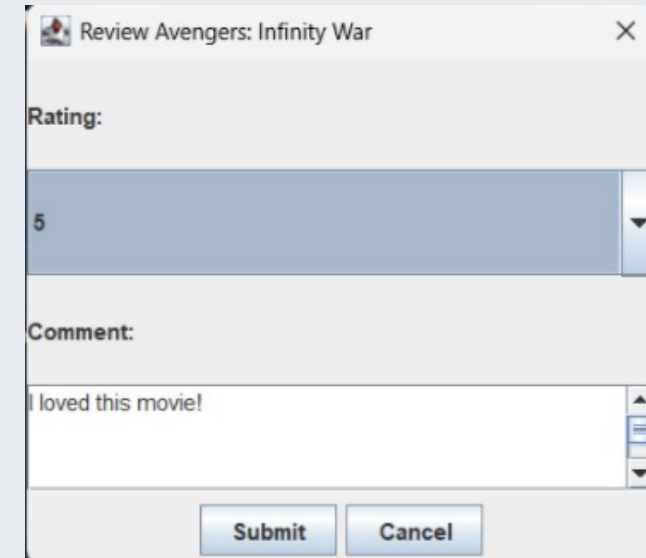
Watch a movie → Give stars → Share thoughts

👤 View Profile

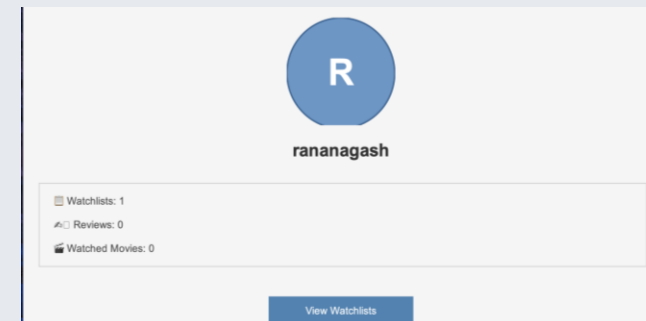
See all your ratings, watchlists, and watch history in one place

📖 Record & View Watch History

Track everything you've watched, rediscover old favourites



A screenshot of a mobile app dialog box titled "Review Avengers: Infinity War". It features a "Rating:" section with a blue bar showing the number "5". Below this is a "Comment:" text area containing the text "I loved this movie!". At the bottom right are "Submit" and "Cancel" buttons.



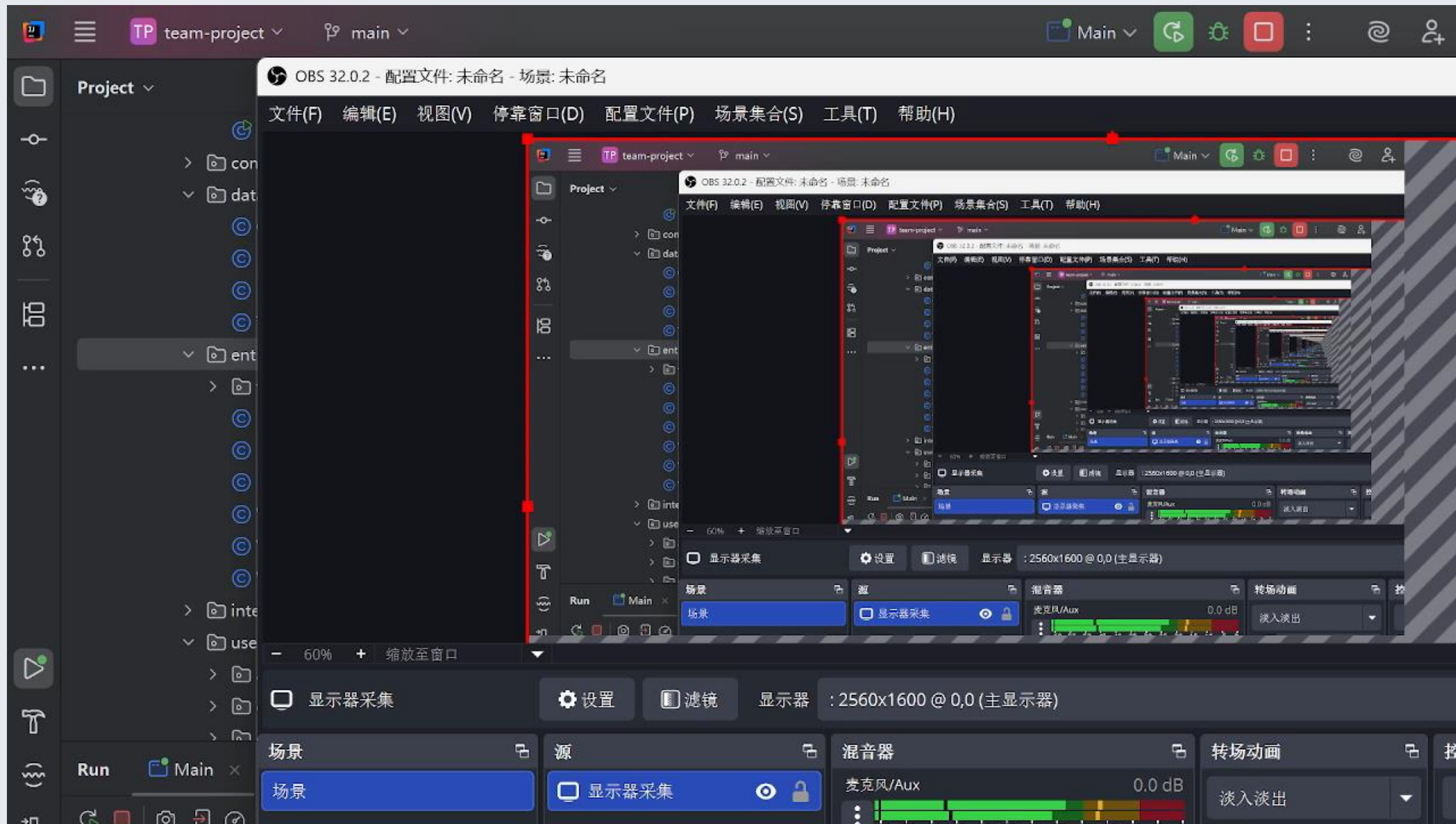
A screenshot of a user profile page for "rananagash". It shows a blue circular profile picture with the letter "R". Below the name, there are statistics: "Watchlists: 1", "Reviews: 0", and "Watched Movies: 0". A blue button labeled "View Watchlists" is at the bottom.

Demo Outline - MVP Core Functionality

- **Search for Movies (Chester):** Search by title, show paginated results with posters and basic info, and open a detailed movie popup.
- **Add to Watchlist (Alana):** From search results or movie detail popup, add a movie to a watchlist and confirm it was added.
- **Add Watch History (Jiaqi):** From the movie detail popup, mark a movie as watched
- **View Watch History (Jiaqi):** From the Profile page, open “View Watch History” to see all watched movies with watched dates, user rating, and user review. Edit popup to change watched date, rating, and review, or delete a movie.
- **Filter by Genre (Inba):** Filter the current movie results by genre to narrow down what to watch.
- **Compare Watchlists (Rana):** Compare watchlists between users to find overlapping movies for group viewing.
- **Rate and Review (Oliver):** After watching, create or update a rating and text review for a movie from the history or profile flows.

Focus areas:

- Search → Record as watched → View / Edit history flow
- User-centric data: personal ratings, reviews, and watched dates
- UI feedback: popups for success / errors, confirmation dialogs, and consistent buttons
- Team features working together: search, watchlists, history, and social comparison





API usage -> TMDB API

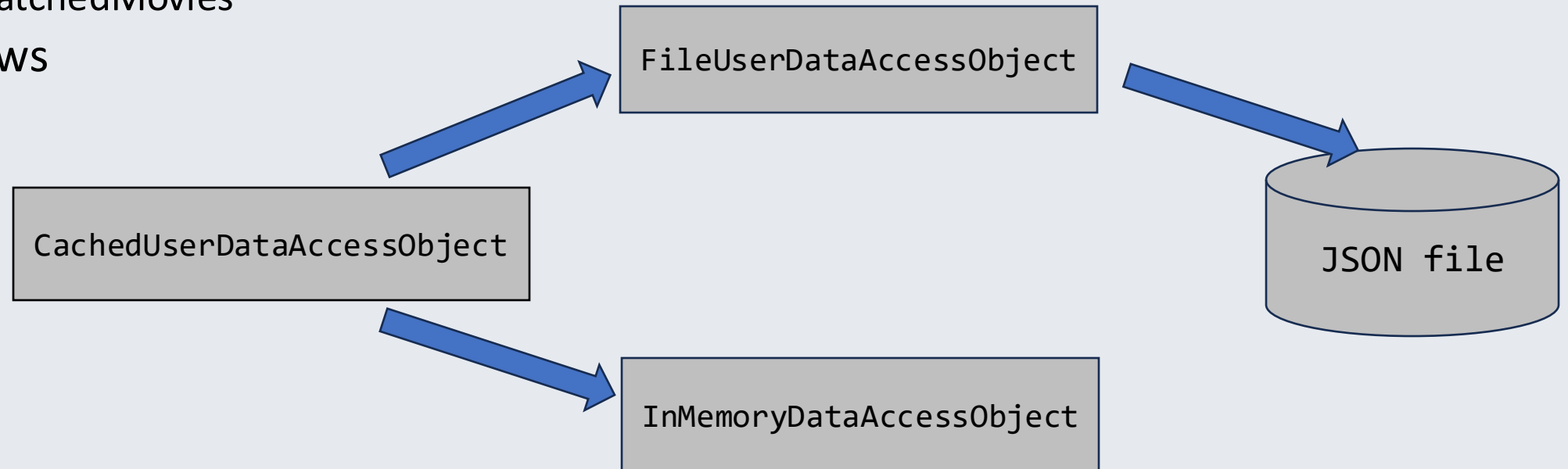


- Used TMDB API only
- Provides titles, posters, genres, release dates, search results
- Chosen for reliability, strong documentation, easy integration
- Supports keyword search for search use cases
- Offers consistent movie IDs used for ratings and reviews
- Fits clean architecture since data arrives as simple JSON

Data Persistence

User

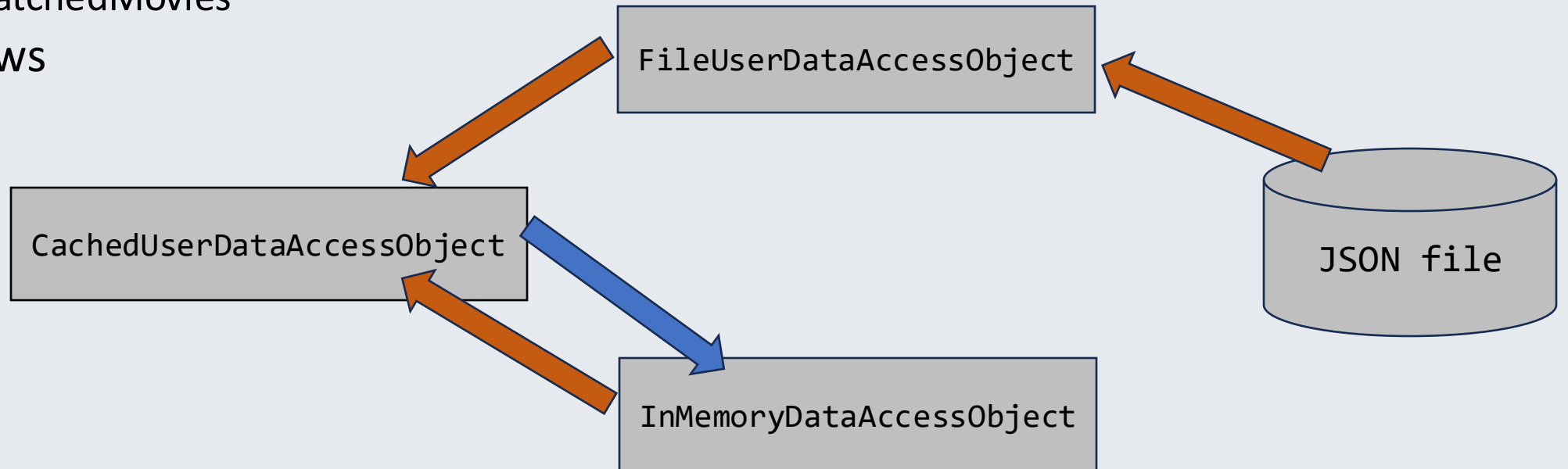
- WatchLists
 - Movies
- WatchHistory
 - WatchedMovies
- Reviews



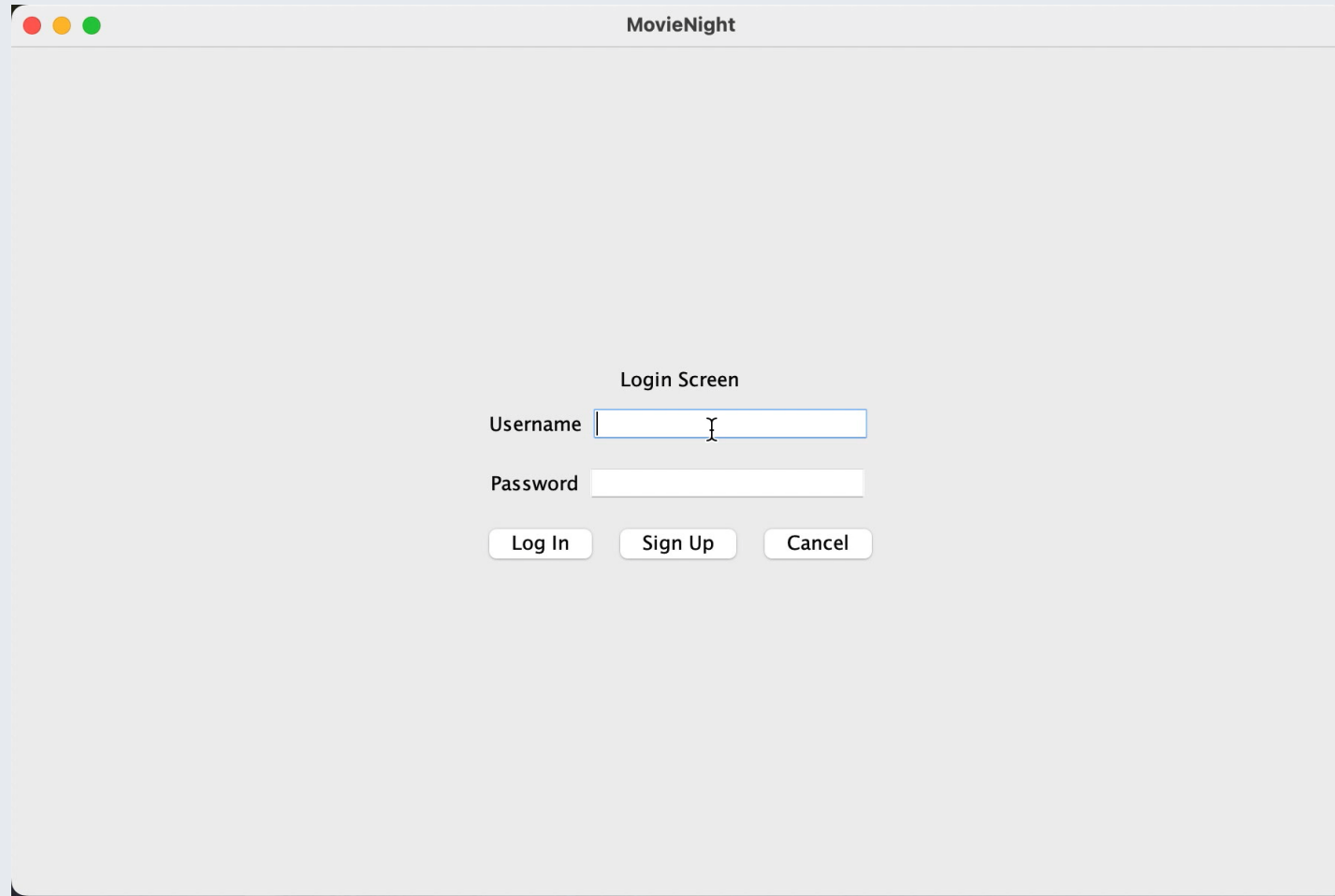
Data Persistence

User

- WatchLists
 - Movies
- WatchHistory
 - WatchedMovies
- Reviews



Data Persistence - Demo



A screenshot of a web application window titled "MovieNight". The window has a light gray background and a title bar with three colored buttons (red, yellow, green) on the left. In the center of the window, there is a "Login Screen" section. It contains two input fields: "Username" and "Password". The "Username" field is currently active, with a cursor inside. Below the input fields are three buttons: "Log In", "Sign Up", and "Cancel".

MovieNight

Login Screen

Username

Password

-Searching Movies-

User Case Focus: Searching Movies

User Story: As a user, I wish to search for movies by title so I can find specific movies or discover new ones.

Photo of Before

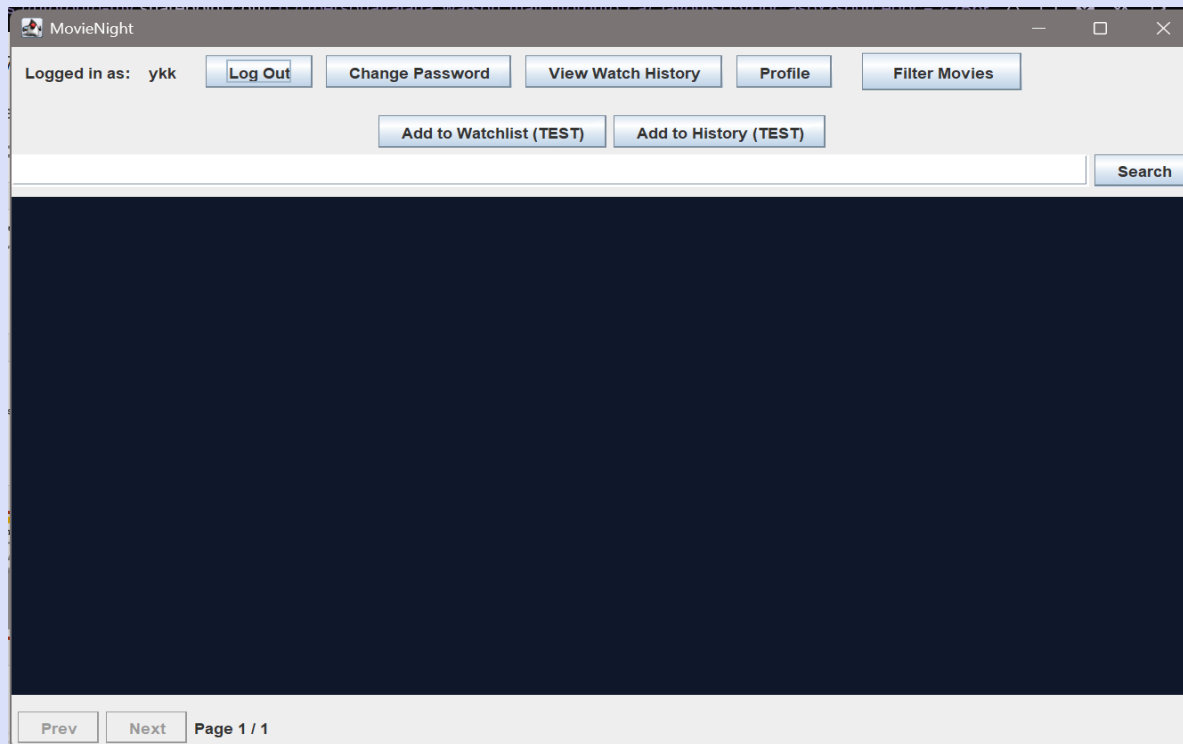
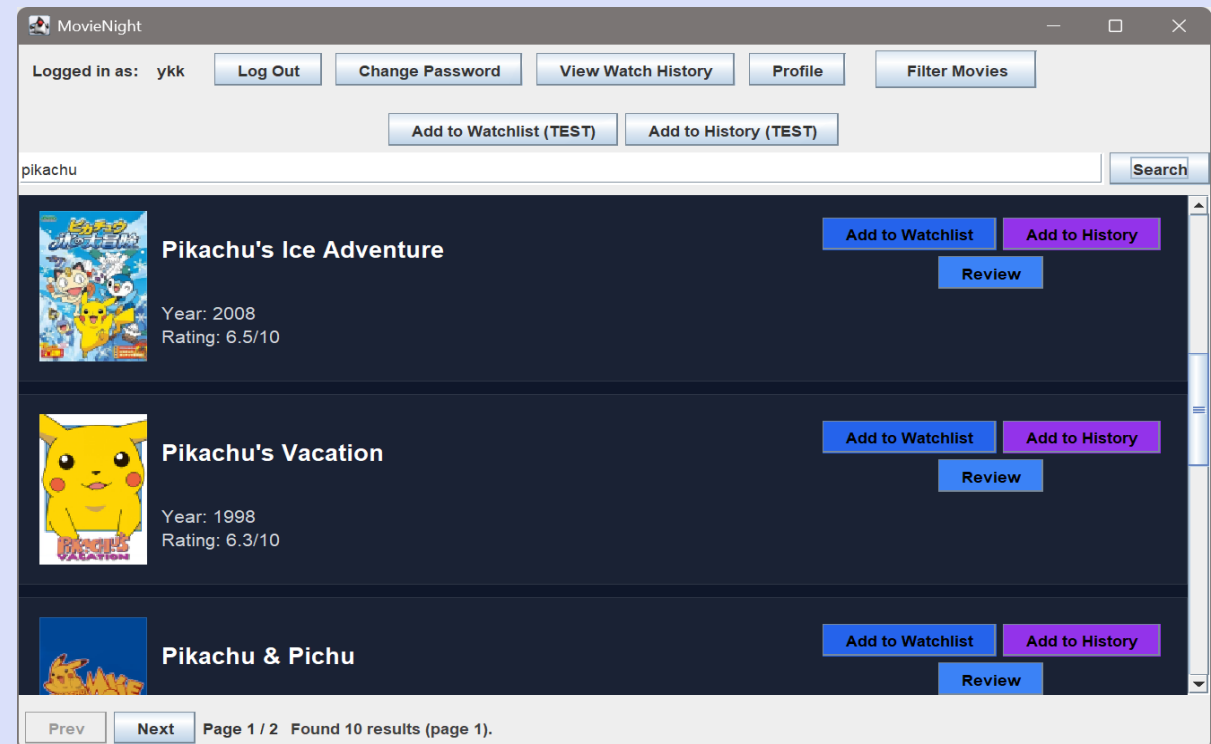
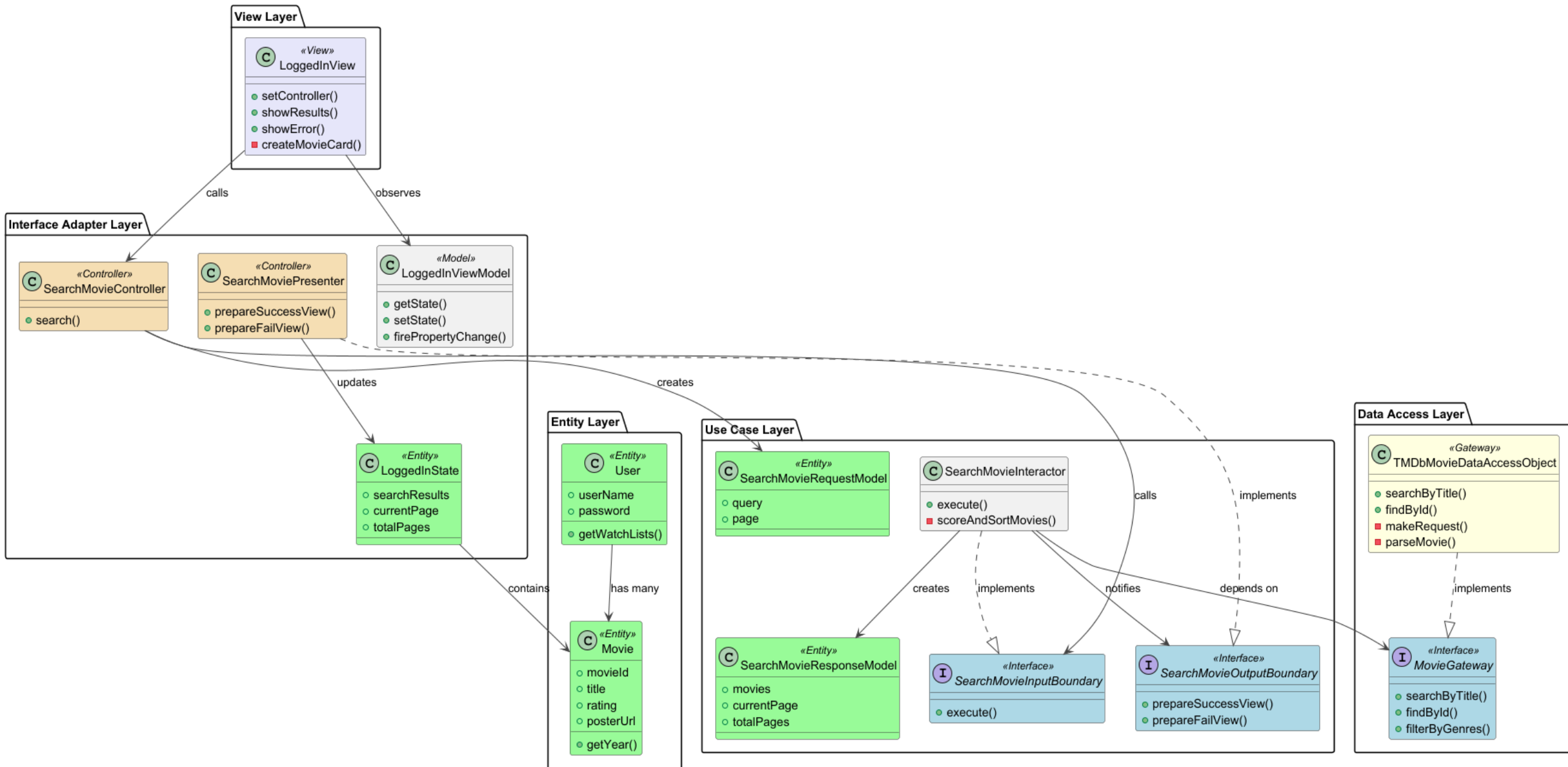


Photo of After



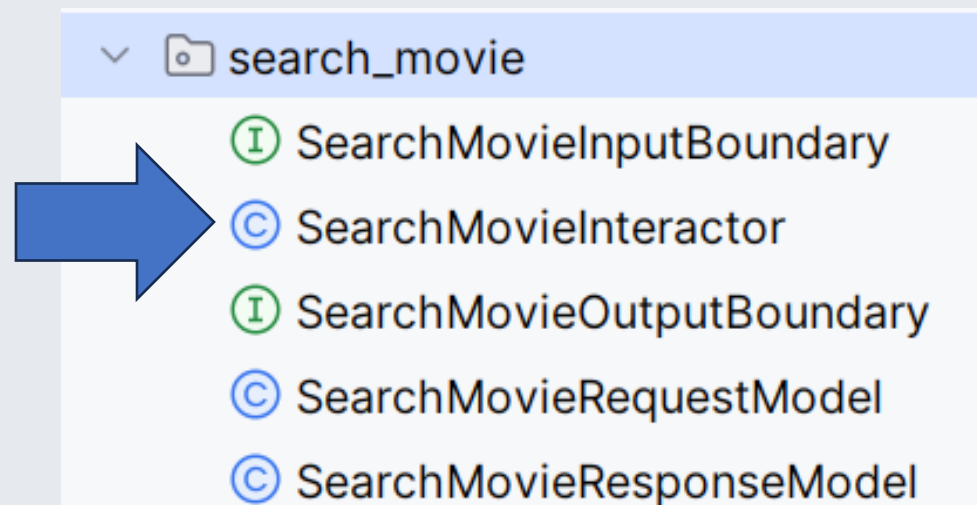
User Case Focus: Searching Movies

Search Feature - Clean Architecture



Use Case Interactor class code

First Photo of Interactor Code



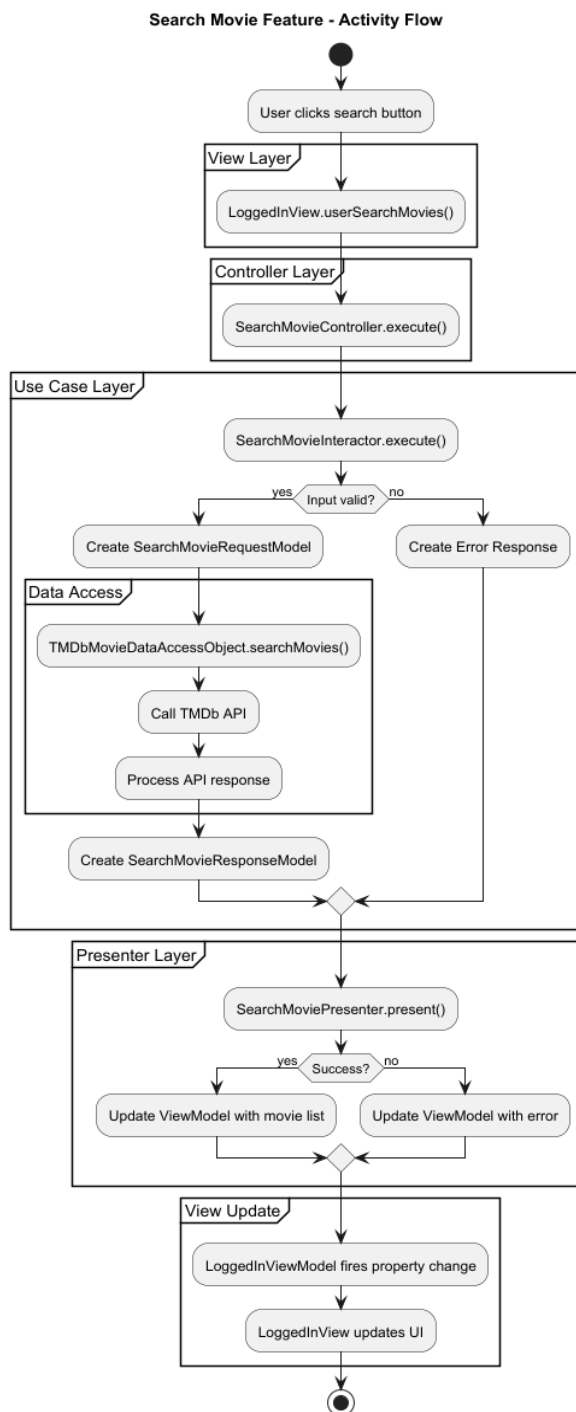
- **Implements** SearchMovieInputBoundary
- **Validates the search query** to ensure it is not null, empty, or too short before proceeding
- **Checks pagination parameters** to guarantee a valid page number is requested.
- **Handles empty or invalid results** gracefully by preparing a user-friendly fail view via the output boundary.

- **Executes the proprietary** scoreAndSortMovies() **algorithm**, which combines:
 - Title match relevance (exact, prefix, contains).
 - Movie rating and popularity.
 - Release year recency.

Second Photo of Interactor Code

```
public class SearchMovieInteractor implements SearchMovieInputBoundary {  
  
    private final MovieGateway movieGateway; 2 usages  
    private final SearchMovieOutputBoundary presenter; 7 usages  
  
    public SearchMovieInteractor(MovieGateway movieGateway, 1 usage 2 Rom  
                                SearchMovieOutputBoundary presenter) {  
        this.movieGateway = movieGateway;  
        this.presenter = presenter;  
    }  
}
```

Flow of Control when Executed



Part 1: User Trigger

- User clicks the search button → starts the flow.

Part 2: View Layer

- `LoggedInView` gets input and calls `userSearchMovies()` — just display + input.

Part 3: Controller

- `SearchMovieController` is the middleman: forwards requests from view to business logic.

Part 4: Business Logic Core

- `SearchMovieInteractor` validates input, then if valid: create request, call data access, create response.

Part 5: Data Access Layer

- `TMDbMovieDataAccessObject` calls the API and converts data; business layer only knows the interface.

Part 6: Presenter Layer

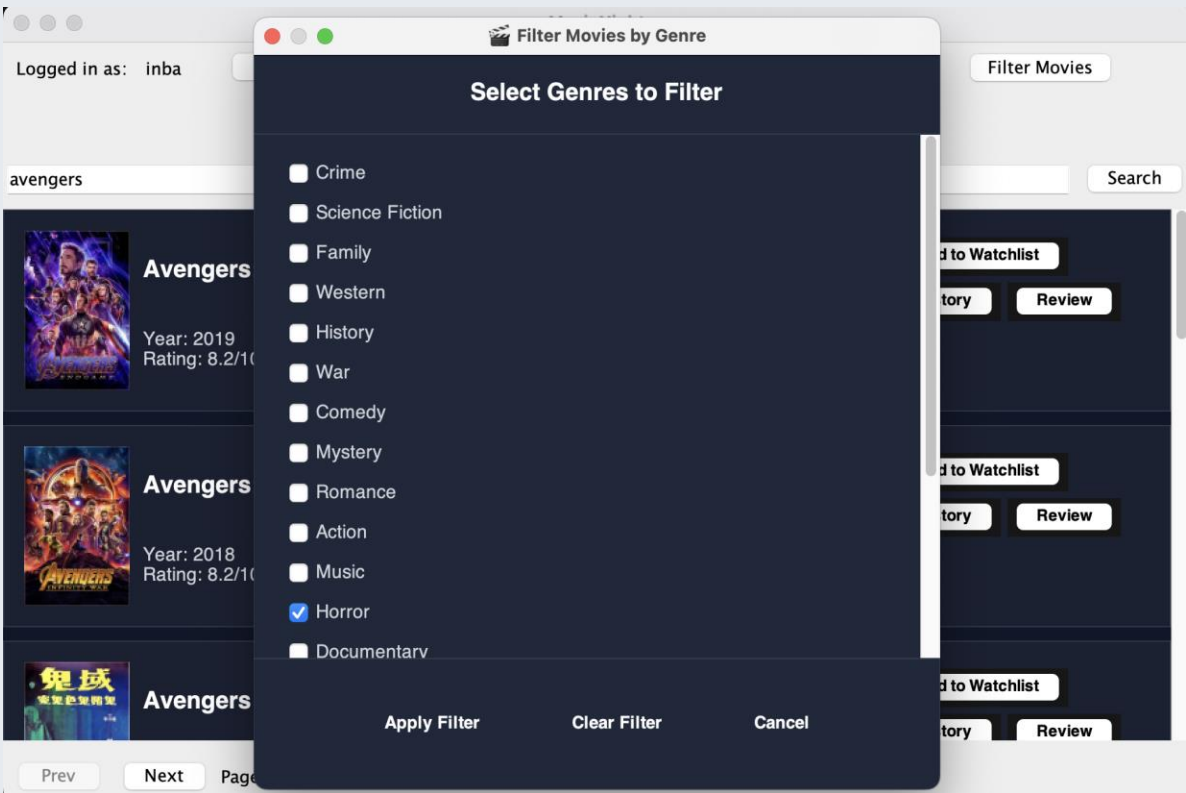
- `SearchMoviePresenter` turns business data into view data; success updates movies, failure shows error.

-Filter Movies by Genre-

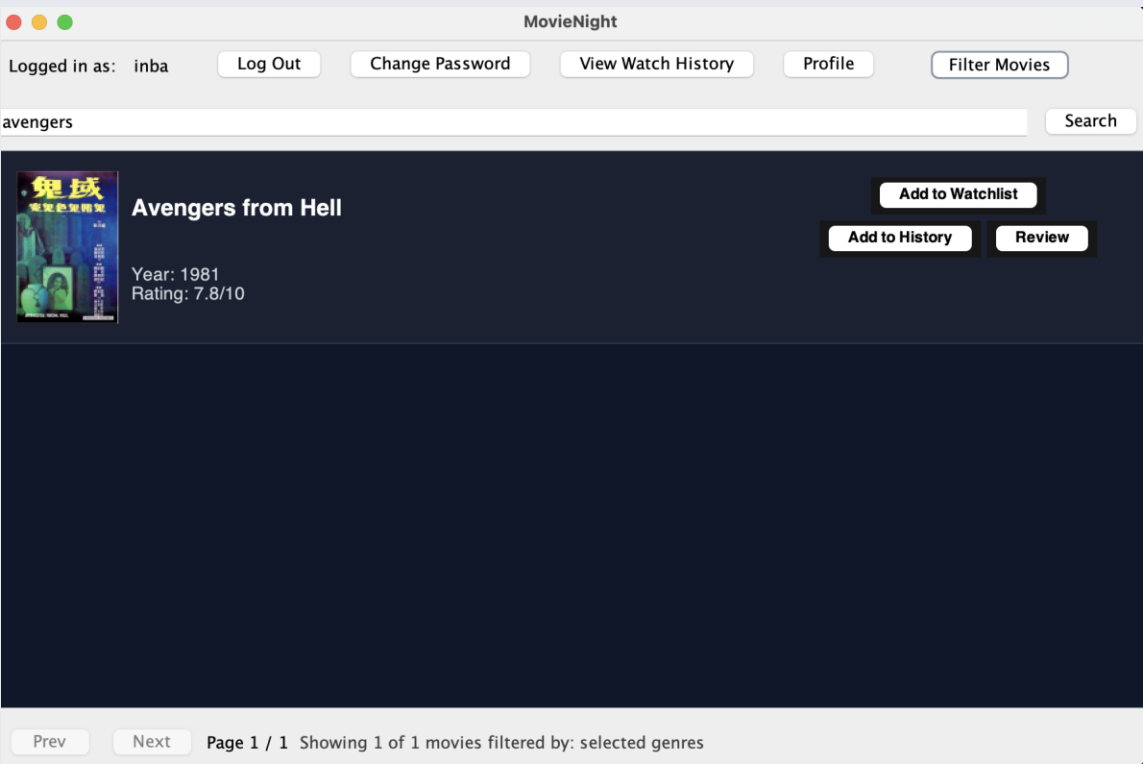
User Case Focus: Filter Movies by Genre

User Story: As a user, I wish to be able to filter movies by genre, so that I can easily find movies that I'm interested in.

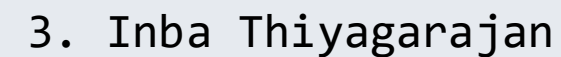
Before View



After View



- UML Class Diagram -



Use Case Interactor class code

```
@Contract(pure = true)
public FilterMoviesInteractor(MovieGateway movieGateway, 2 usages 2 inbathiyagarajan +1
                             FilterMoviesOutputBoundary presenter,
                             FilterMoviesValidator validator,
                             MovieFilterStrategy filterStrategy,
                             GenreConverter genreConverter) {

    this.movieGateway = movieGateway;
    this.presenter = presenter;
    this.validator = validator;
    this.filterStrategy = filterStrategy;
    this.genreConverter = genreConverter;
}
```

- Validates Input: Checks if the user's request is correct and complete. If something is wrong, immediately shows an error message using the presenter.
- Filter Movies: Uses a flexible filtering strategy to find movies matching the selected genres.
- Convert Genres: Turns internal genre IDs into readable genre names so users can see them clearly.
- Prepare Response: Wraps the filtered movies and genre names into a response object. Sends it to the presenter to display the results to the user.

- The interactor receives all required collaborators through its constructor for loose coupling and easier testing.
- Each dependency handles a distinct responsibility (gateway, presenter, validator, filter strategy, genre conversion).
- The MovieFilterStrategy and GenreConverter allow swapping logic without modifying core use-case code.
- Interactor depends only on abstractions, for a stable and testable use-case layer (follows Clean Architecture).

```
@Override 2 inbathiyagarajan +1
public void execute(FilterMoviesRequestModel requestModel) {
    // Validate request
    String validationError = validator.validate(requestModel);
    if (validationError != null) {
        presenter.prepareFailView(validationError);
        return;
    }

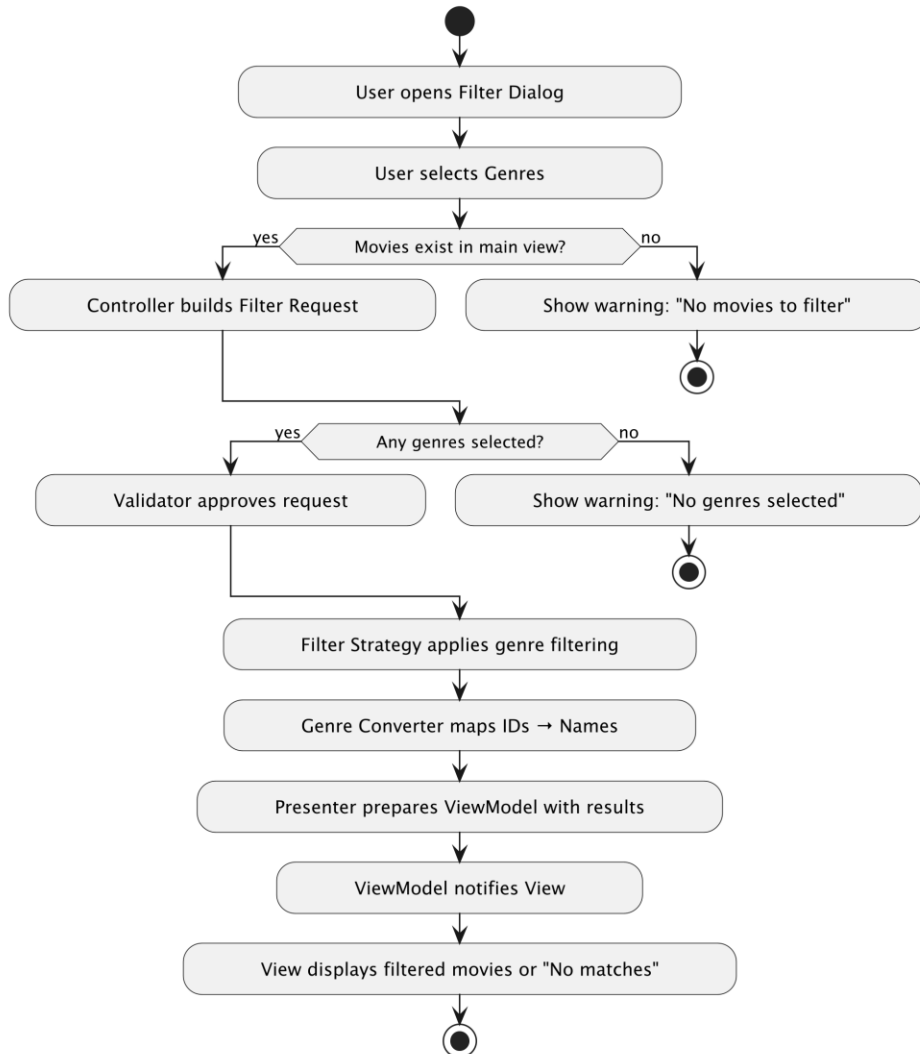
    // Filter movies using the configured strategy
    List<Movie> filteredMovies = filterStrategy.filter(
        requestModel.getMoviesToFilter(),
        requestModel.getGenreIds());

    // Convert genre IDs to human-readable genre names
    List<String> genreNames = genreConverter.convertGenreIdsToNames(
        requestModel.getGenreIds());

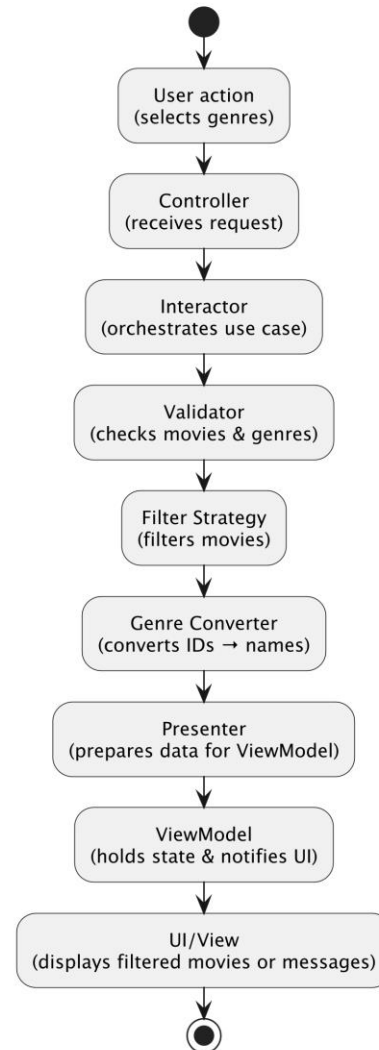
    // Prepare the response for the presenter
    presenter.prepareSuccessView(new FilterMoviesResponseModel(
        requestModel.getGenreIds(),
        genreNames,
        filteredMovies));
}
```

Flow of Control when Executed

Filter Movies – Wide Flow of Control Diagram



Filter Movies – Clean Architecture Flow



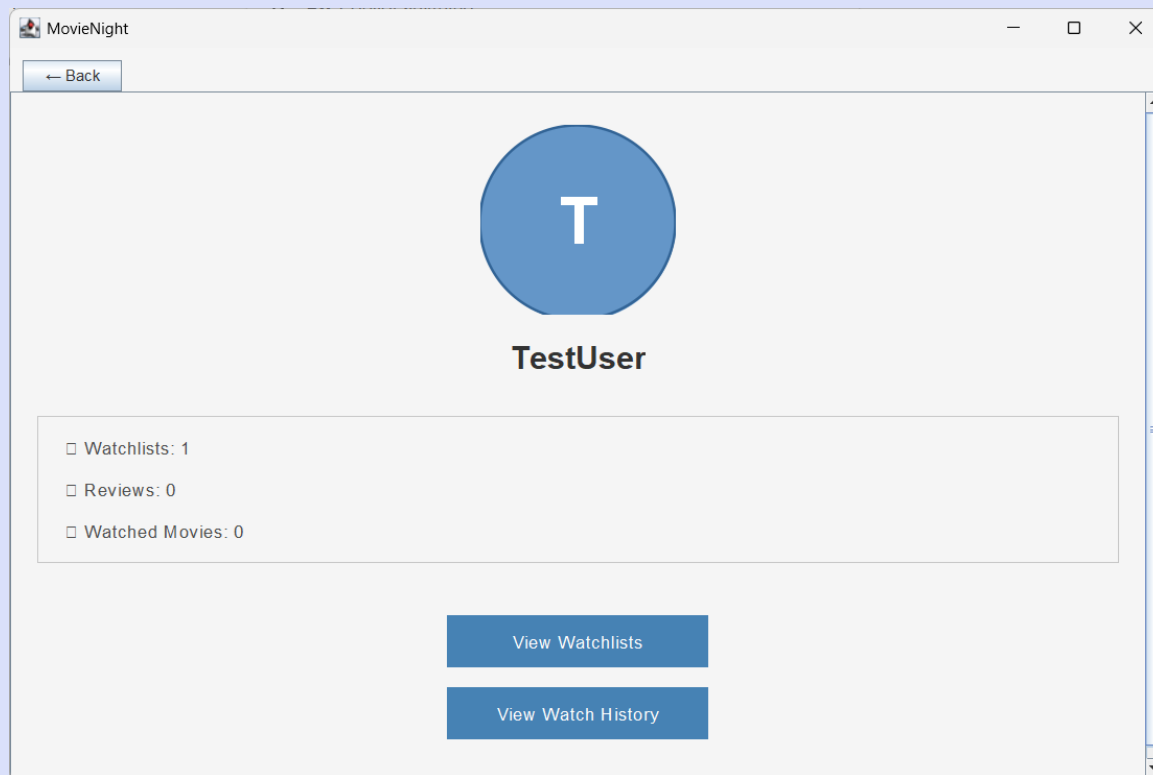
- User opens filter dialog and selects genres.
- System checks for movies and selected genres, shows warnings if needed.
- Filter strategy applies genres → Presenter (prepares results) → View (displays filtered) movies.
- Process ends with either results or a “No matches” message.

-View Movie WatchLists-

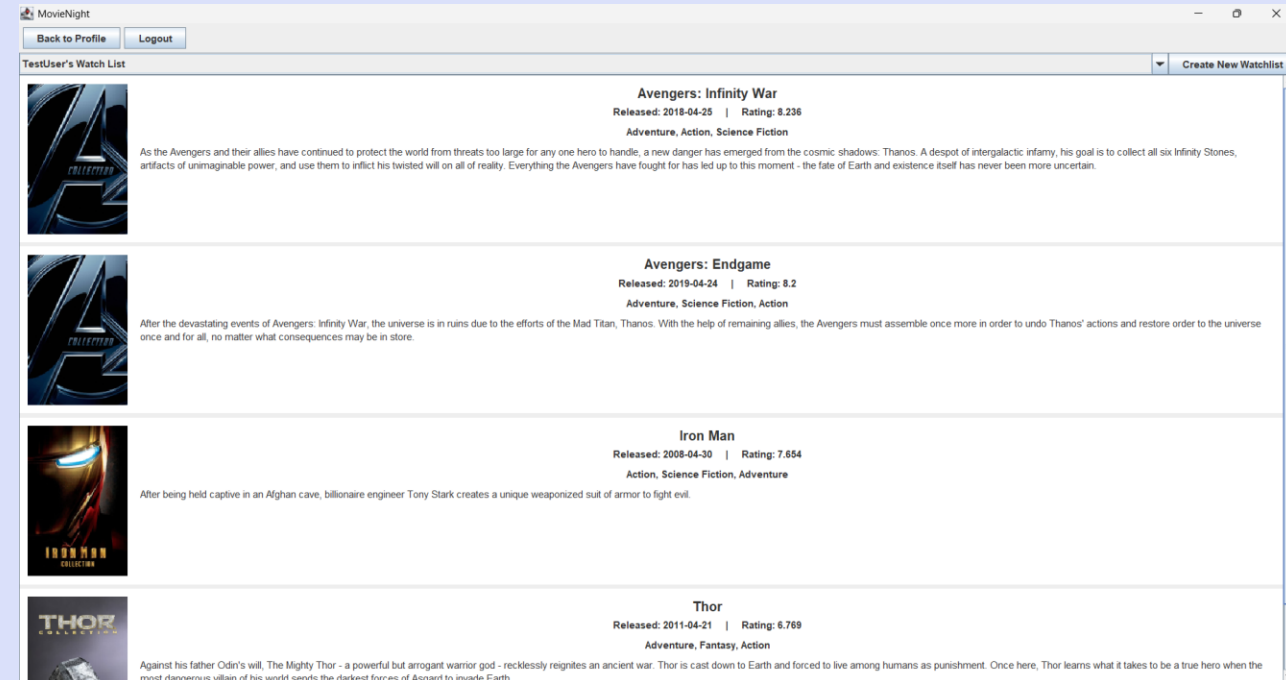
User Case Focus: View Movie WatchLists

User Story: As a user, I wish to view my watchlists of movies so that I can keep track of movies to watch later.

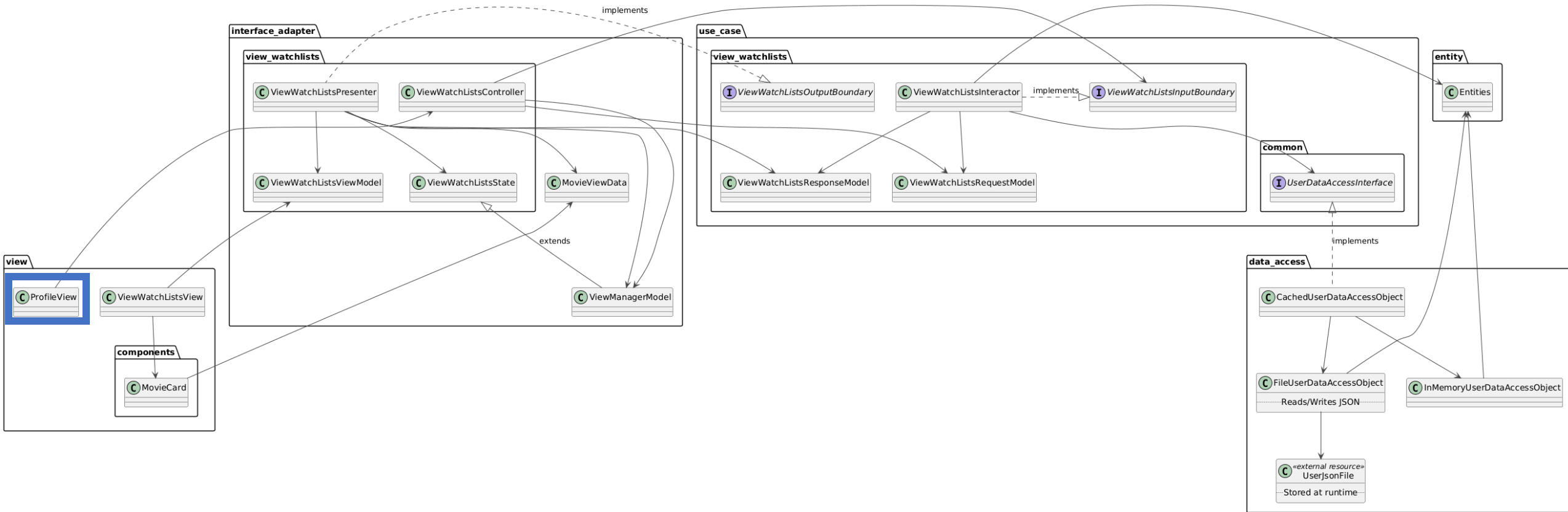
View Before



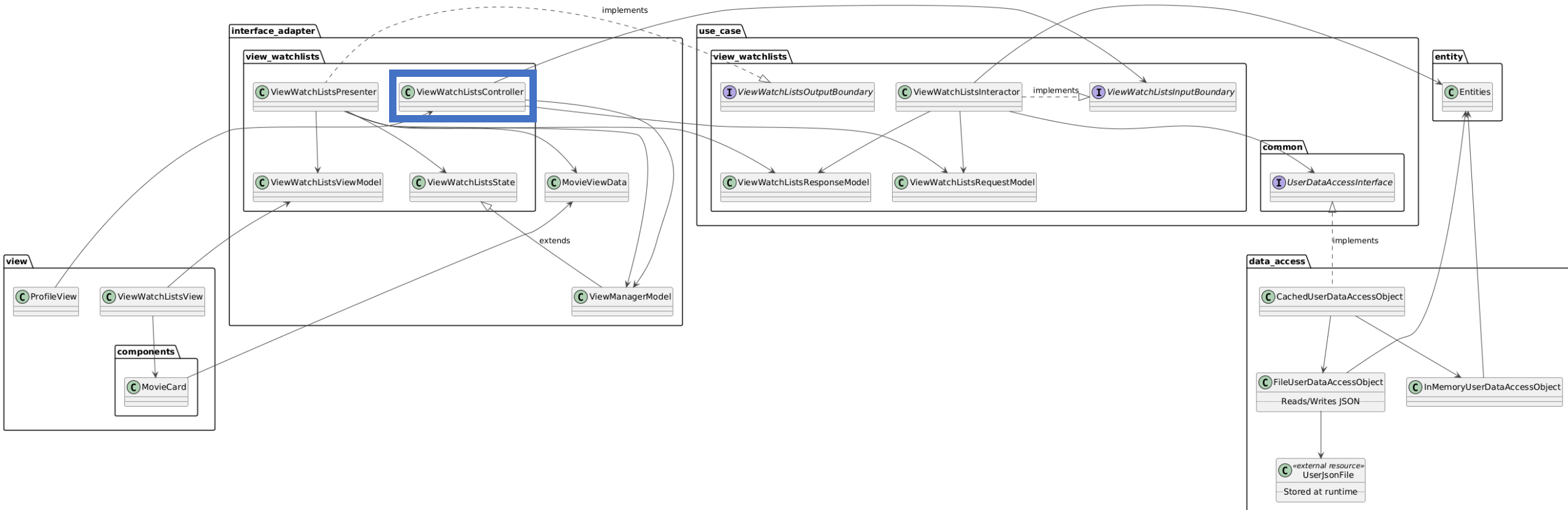
View After



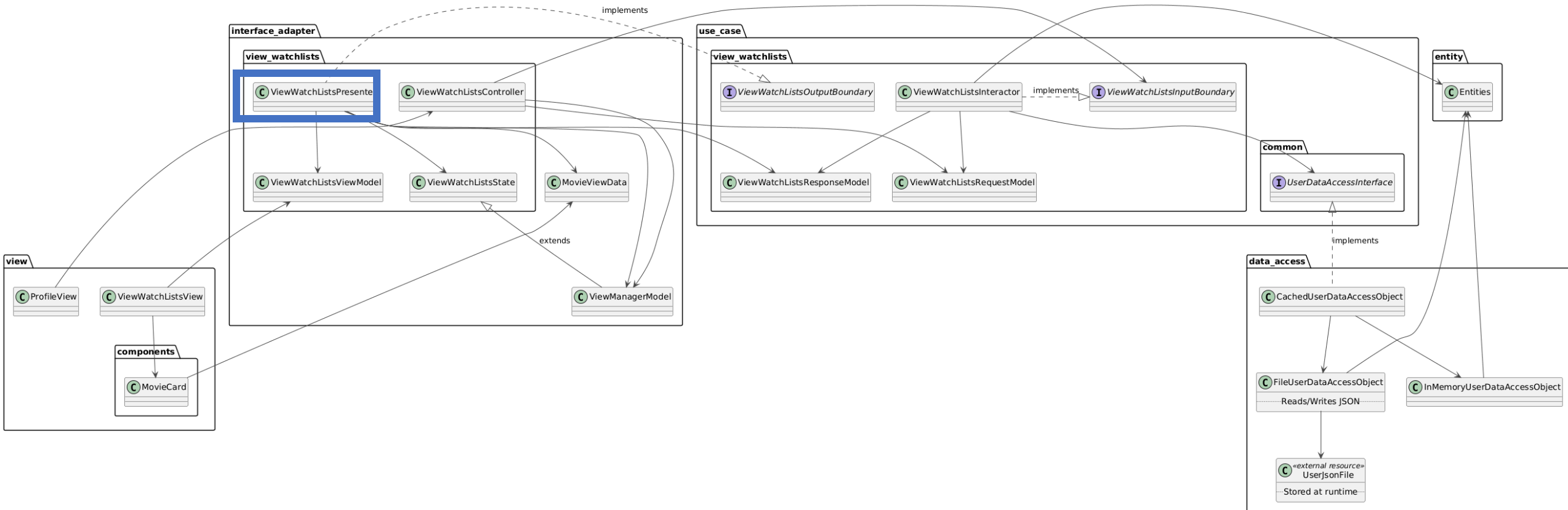
- UML Class Diagram -



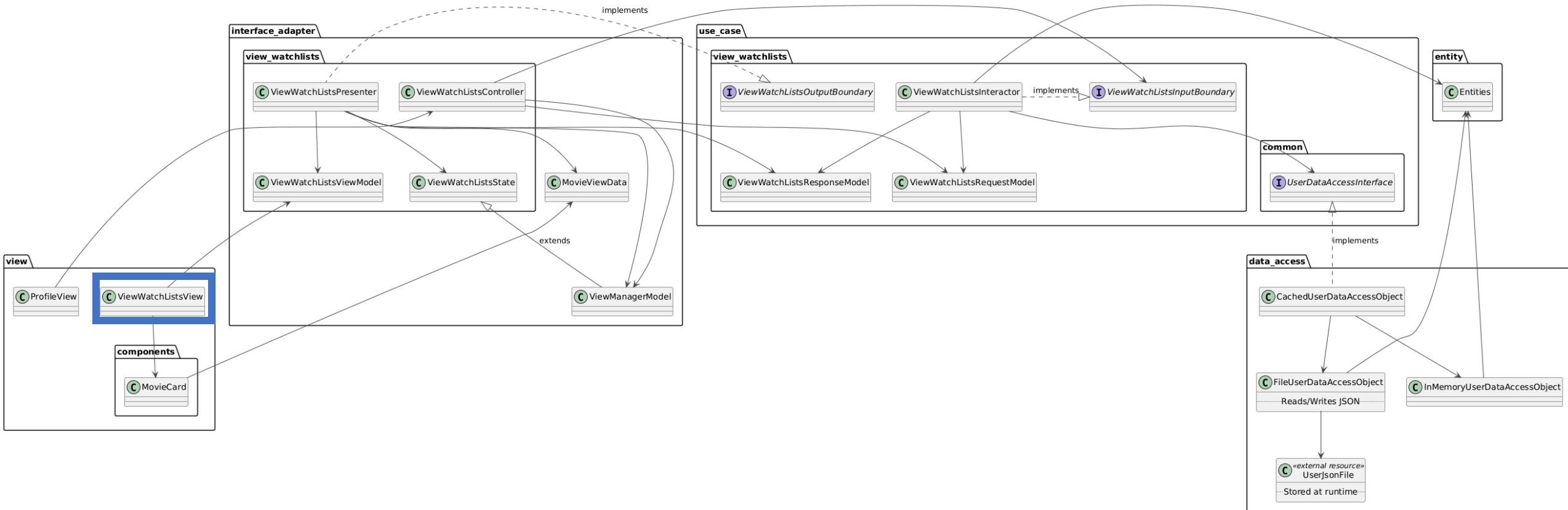
- UML Class Diagram -



- UML Class Diagram -



- UML Class Diagram -



Use Case Interactor class code

- Checks the user exists
- Checks if they have any watchlists
- Gathers a list of their watchlists
- Finds out which watchlist to look at first

- Gathers the movies in the watchlists
- Sends the information to the presenter

```
public void execute(ViewWatchListsRequestModel requestModel) {
    final String username = requestModel.getUsername();
    final User user = userDataAccess.getUser(username);
    if (user == null) {
        presenter.prepareFailureView( errorMessage: "User not found.");
        return;
    }

    final List<WatchList> lists = user.getWatchLists();
    if (lists.isEmpty()) {
        presenter.prepareFailureView( errorMessage: "User has no watchlists.");
        return;
    }

    final List<ViewWatchListsResponseModel.WatchListInfo> listInfos = new ArrayList<>();
    for (WatchList watchList : lists) {
        listInfos.add(new ViewWatchListsResponseModel.WatchListInfo(
            watchList.getWatchListId(),
            watchList.getName()
        ));
    }

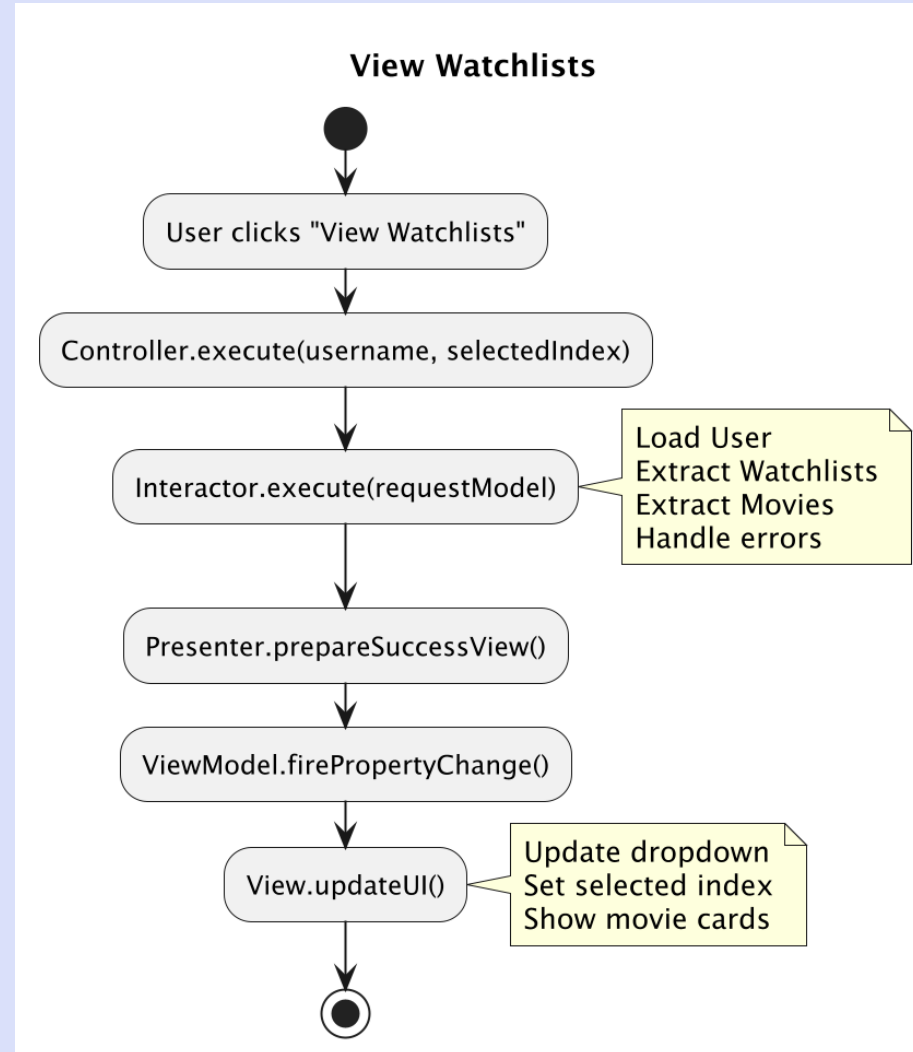
    int index = requestModel.getSelectedIndex();
    if (index < 0 || index >= listInfos.size()) {
        index = 0;
    }

    final WatchList selected = lists.get(index);
```

```
final List<ViewWatchListsResponseModel.MovieInfo> movies = new ArrayList<>();
for (Movie movie : selected.getMovies()) {
    movies.add(new ViewWatchListsResponseModel.MovieInfo(
        movie.getMovieId(),
        movie.getTitle(),
        movie.getPlot(),
        movie.getGenreIds(),
        movie.getReleaseDate(),
        movie.getRating(),
        movie.getPosterUrl()
    ));
}

presenter.prepareSuccessView(new ViewWatchListsResponseModel(
    listInfos,
    index,
    movies,
    username
));
```

Flow of Control when Executed

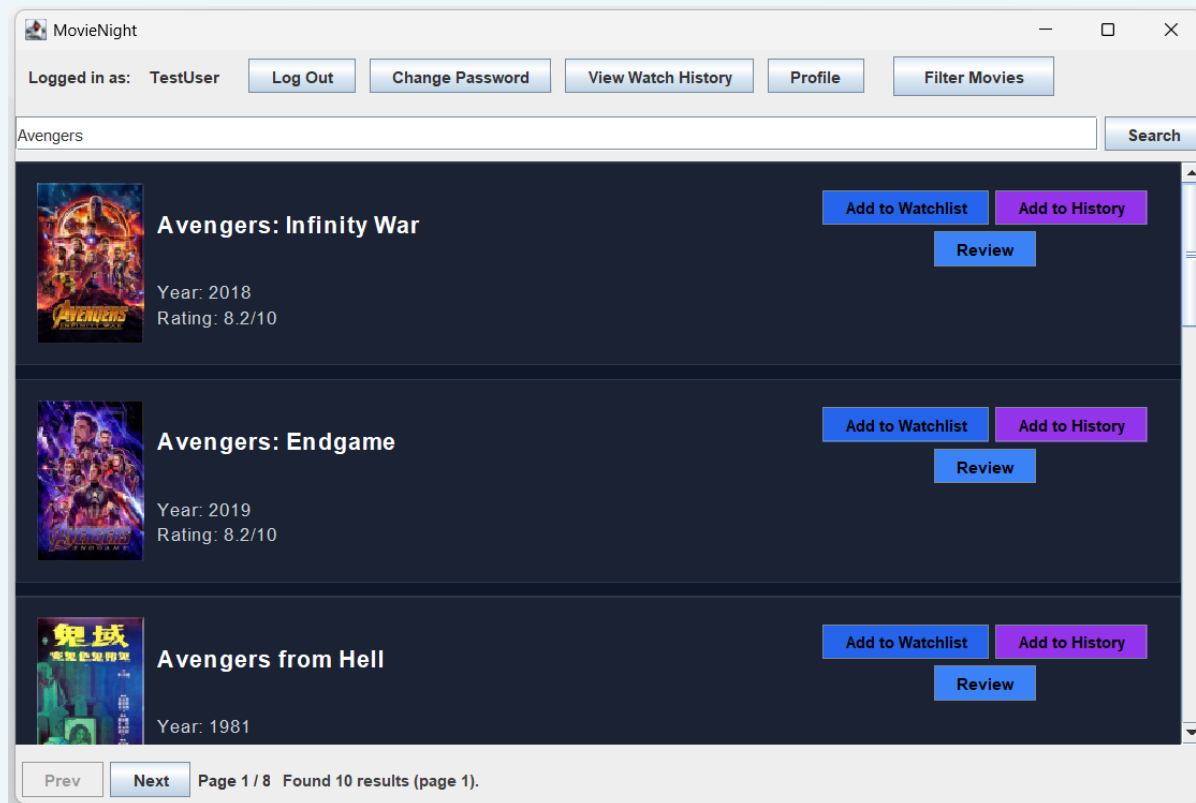


-Rating & Reviewing Movies-

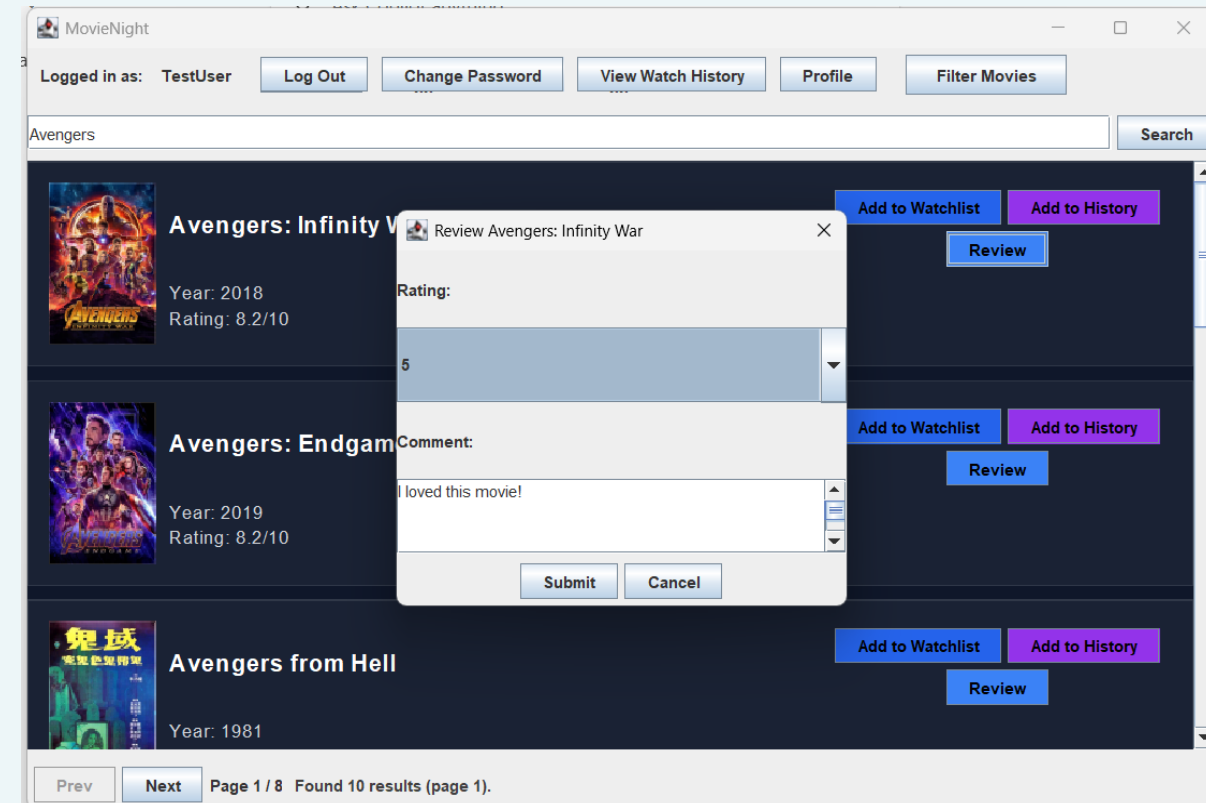
Rating & Reviewing Movies

User Story: As a user, I wish to be able to rate and review movies I've watched so that others can see my opinions and get recommendations

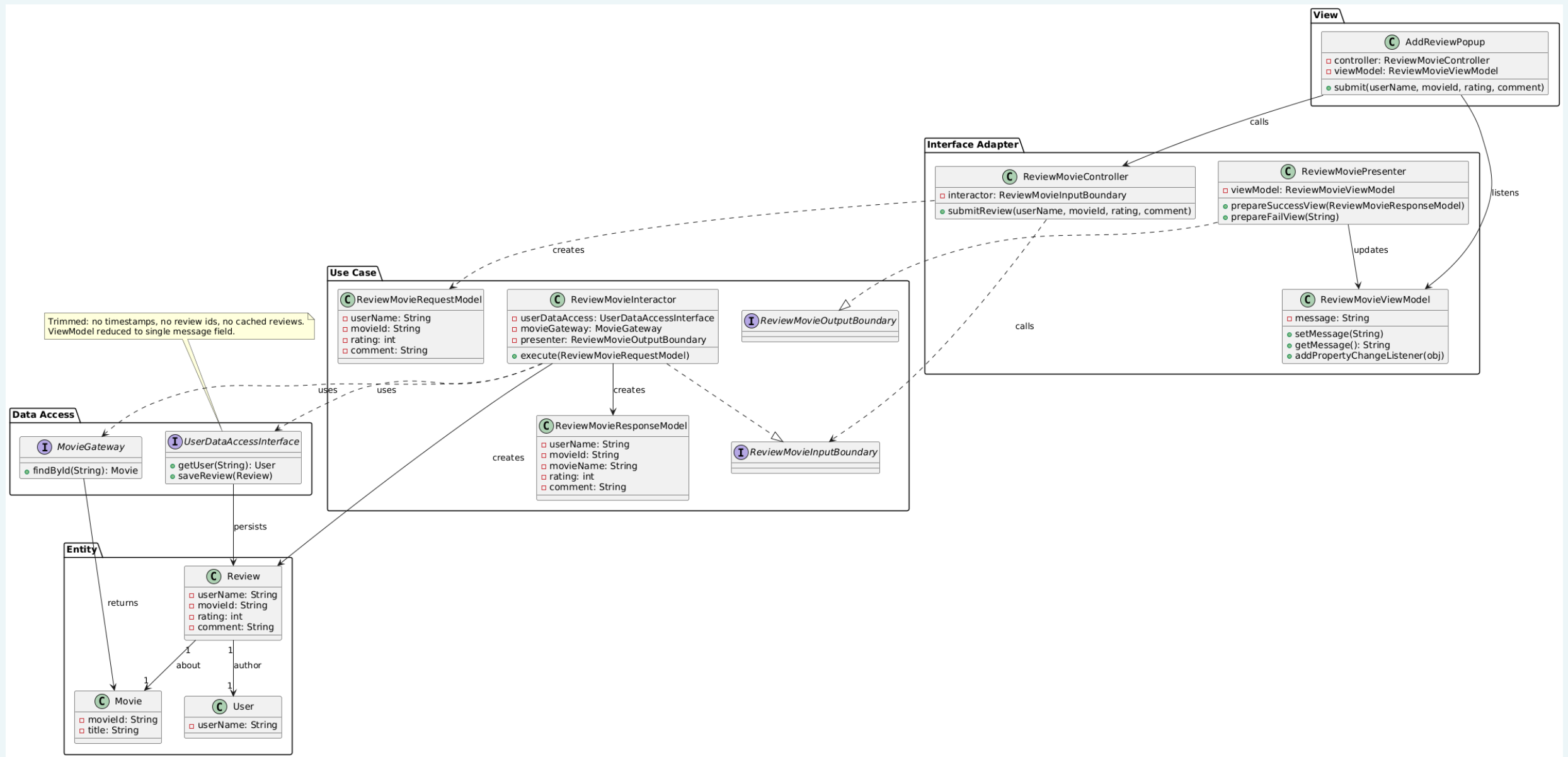
Before View



After View



- UML Class Diagram -



Use Case Interactor class code

- Uses MovieGateway to fetch movie by ID
- Creates a Review entity with UUID and timestamp
- Sends results to output boundary, letting the presenter format a success/failure view: keeping the interactor independent of presentation.

```

1 package use_case.review_movie;
2
3 > import java.util.UUID;
4
11
12 public class ReviewMovieInteractor implements ReviewMovieInputBoundary {
13
14     private final UserDataAccessInterface userDataAccess;
15     private final MovieGateway movieGateway;
16     private final ReviewMovieOutputBoundary presenter;
17
18     public ReviewMovieInteractor(UserDataAccessInterface userDataAccess,
19                                 MovieGateway movieGateway,
20                                 ReviewMovieOutputBoundary presenter) {
21         this.userDataAccess = userDataAccess;
22         this.movieGateway = movieGateway;
23         this.presenter = presenter;
24     }
25
26     @Override
27     public void execute(ReviewMovieRequestModel requestModel) {
28         if (requestModel.getRating() < 1 || requestModel.getRating() > 5) {
29             presenter.prepareFailView(errorMessage: "Rating must be between 1 and 5.");
30             return;
31         }
32
33         User user = userDataAccess.getUser(requestModel.getUserName());
34         if (user == null) {
35             presenter.prepareFailView(errorMessage: "User not found: " + requestModel.getUserName());
36             return;
37         }

```

- Implements ReviewMovieInputBoundary to follow Clean Architecture
- Validates rating input and checks for null user/movie
- Uses UserDataAccessInterface to fetch and save user data

```

33         User user = userDataAccess.getUser(requestModel.getUserName());
34         if (user == null) {
35             presenter.prepareFailView(errorMessage: "User not found: " + requestModel.getUserName());
36             return;
37         }
38
39         Movie movie = movieGateway.findById(requestModel.getMovieId()).orElse(null);
40         if (movie == null) {
41             presenter.prepareFailView(errorMessage: "Movie not found: " + requestModel.getMovieId());
42             return;
43         }
44
45         Review review = new Review(UUID.randomUUID().toString(), user, movie,
46                                   requestModel.getRating(), requestModel.getComment(), LocalDateTime.now());
47         user.addReview(review);
48         userDataAccess.save(user);
49         presenter.prepareSuccessView(new ReviewMovieResponseModel(
50             review.getReviewId(),
51             user.getUserName(),
52             movie.getMovieId(),
53             movie.getTitle(),
54             review.getRating(),
55             review.getComment());
56     }
57 }
58

```

Flow of Control when Executed

- User clicks the **Review** button on a movie card in `LoggedInView`.
- System opens the **AddReviewPopup**, showing rating and comment fields.
- User enters a **rating (1–5)** and a **comment**, then clicks **Submit**.
- `ReviewMovieController` receives input and calls the **Interactor** with a `ReviewMovieRequestModel`.
- **Interactor validates input**, fetches User and Movie, creates a Review, updates the User, and calls the **Presenter**.
- **Presenter updates the ReviewMovieViewModel** with success or failure, firing property change events.
- `AddReviewPopup` listens to the ViewModel events and displays a **message dialog** with the outcome, then closes.

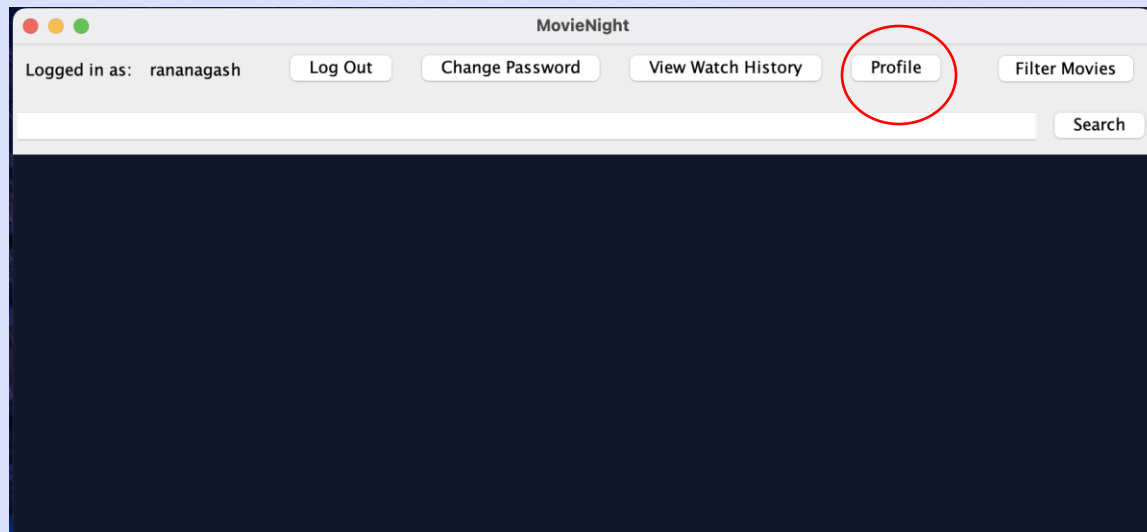
User action → Controller → Interactor → Presenter → ViewModel → UI

-View User Profile-

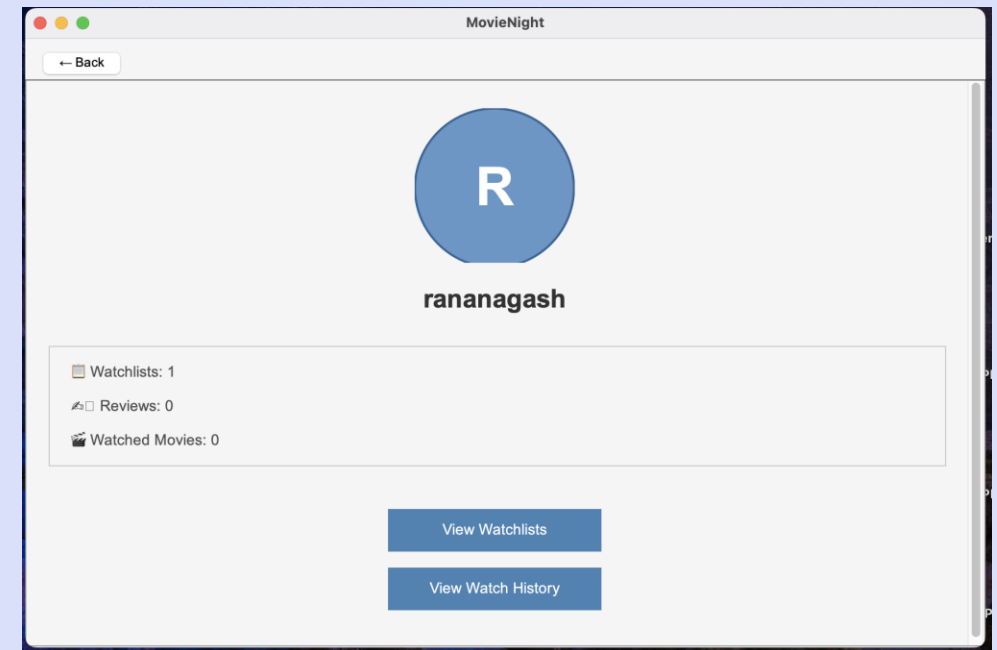
User Case Focus: Viewing Profile

User Story: As a user, I wish to be able to view my profile so that I can see my activity in one place

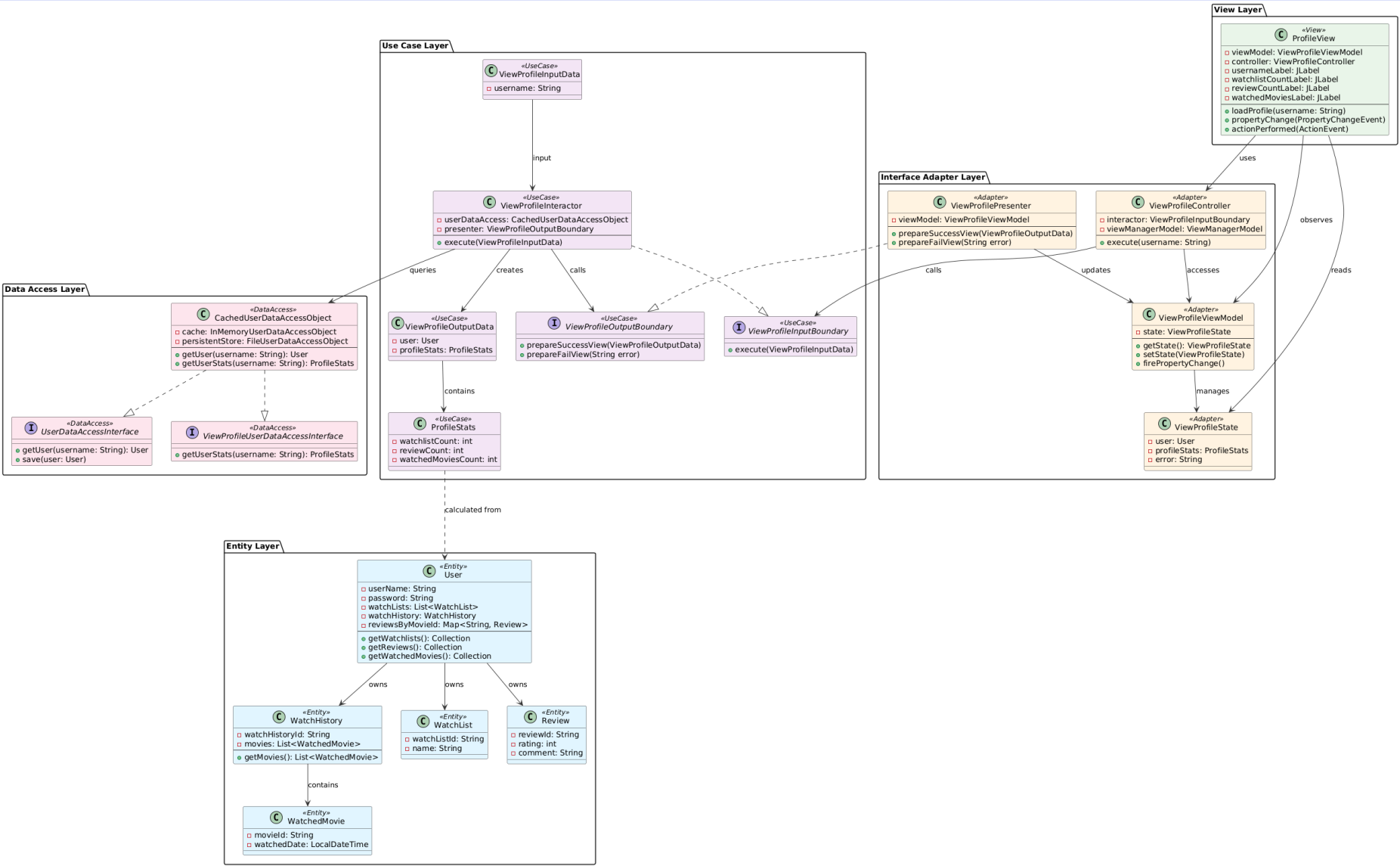
Before View



After View



- UML Class Diagram -



Use Case Interactor Code

```
public class ViewProfileInteractor implements ViewProfileInputBoundary {
    private final CachedUserDataAccessObject userDataAccessObject;
    private final ViewProfileOutputBoundary userPresenter;

    public ViewProfileInteractor(CachedUserDataAccessObject userDataAccessInterface,
                                ViewProfileOutputBoundary viewProfileOutputBoundary) {
        this.userDataAccessObject = userDataAccessInterface;
        this.userPresenter = viewProfileOutputBoundary;
    }
}
```

•Fetches user data

Calls userDataAccessObject.getUser(username) to retrieve User entity.

•Gathers profile statistics

Retrieves ProfileStats via getUserStats(username) for user activity data.

•Packages output

Creates ViewProfileOutputData(user, stats) with complete profile information.

•Delegates to presenter

Calls prepareSuccessView(outputData) to pass results to presentation layer.

•Receives username input data

Accepts ViewProfileInputData containing the username parameter for processing.

•Fetches user from data access layer

Retrieves User entity via CachedUserDataAccessObject.getUser() method.

•Gathers user statistics and profile data

Collects ProfileStats through getUserStats() to complement user information.

•Packages complete profile view

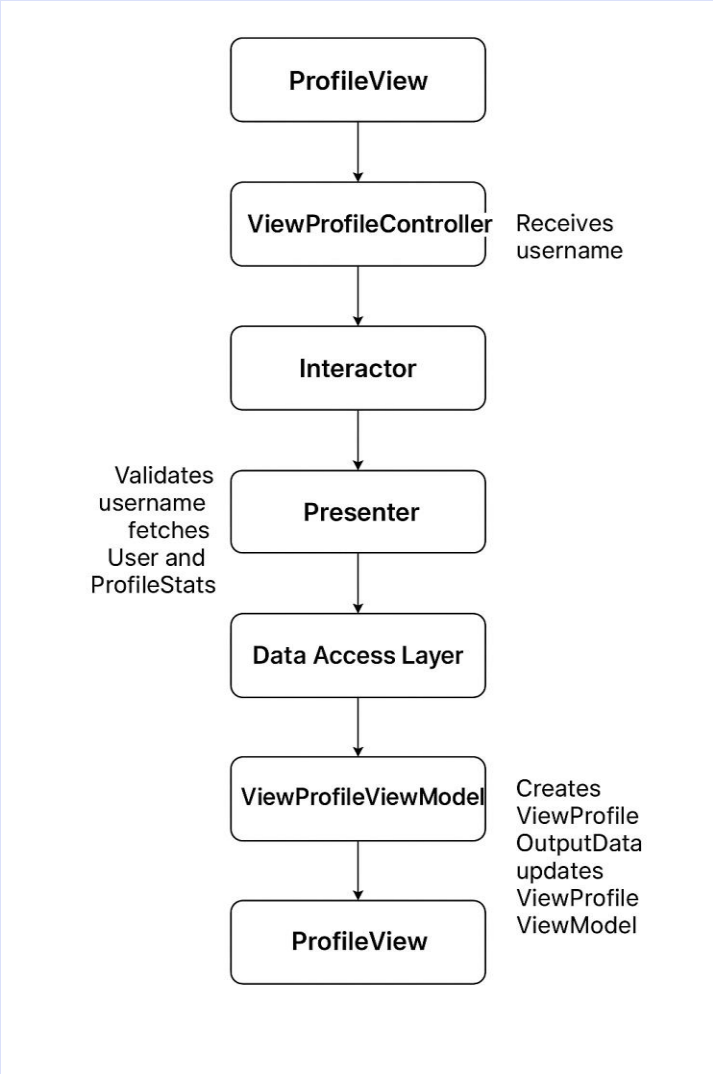
Constructs ViewProfileOutputData containing both user entity and activity statistics.

```
@Override
public void execute(ViewProfileInputData viewProfileInputData) {
    String username = viewProfileInputData.getUsername();

    User user = userDataAccessObject.getUser(username);
    if (user == null) {
        userPresenter.prepareFailView(error: "User not found: " + username);
        return;
    }

    ProfileStats stats = userDataAccessObject.getUserStats(username);
    ViewProfileOutputData outputData = new ViewProfileOutputData(user, stats);
    userPresenter.prepareSuccessView(outputData);
}
```

Flow of Control when Executed



User clicks "Profile" button from LoggedInView.



System displays the ProfileView, showing user information.



ViewProfileController receives the username and calls the Interactor with a ViewProfileInputData.



Interactor validates the username, fetches User and ProfileStats from the data access layer, and calls the Presenter.



Presenter creates ViewProfileOutputData and updates the ViewProfileViewModel with user profile information, firing property change events.



ProfileView listens to the ViewModel events and displays the complete user profile including watch history and rating statistics.

-View Watch History-

User Case Focus: View Watch History

User Story: As a user, I wish to be able to rate and review movies I've watched so that others can see my opinions and get recommendations

After View

Before View

Watch History

×

Watch History - romy

Total: 3 movie(s) watched

1. Batman

Watched: 2025-12-01 00:17

Rating: 4/5 ☐

Review: Not reviewed yet

DetailsDelete

ID: 268

2. Flipped

Watched: 2025-12-01 00:18

Rating: 4/5 ☐

Review: Not reviewed yet

DetailsDelete

ID: 43949

3. Avengers: Endgame

Watched: 2025-12-01 06:13

Rating: Not rated yet

Review: Not reviewed yet

DetailsDelete

ID: 299534

Close

Edit Movie Details

×

Movie Information (Read-only)



Batman

Genres: [14, 28, 80]

Release Date: 1989-06-21

TMDb Rating: 7.2/10

Having witnessed his parents' brutal murder as a child, millionaire philanthropist Bruce Wayne fights crime in Gotham City disguised as Batman, a costumed hero who strikes fear into the hearts of villains. But when a deformed madman known as 'The Joker' seizes control of Gotham's criminal underworld, Batman must face his most ruthless nemesis ever while protecting both his identity and his love interest, reporter Vicki Vale.

Editable Information

Date Watched: 2025-12-01

Time (HH:mm): 00:17

(Format: YYYY-MM-DD HH:mm)

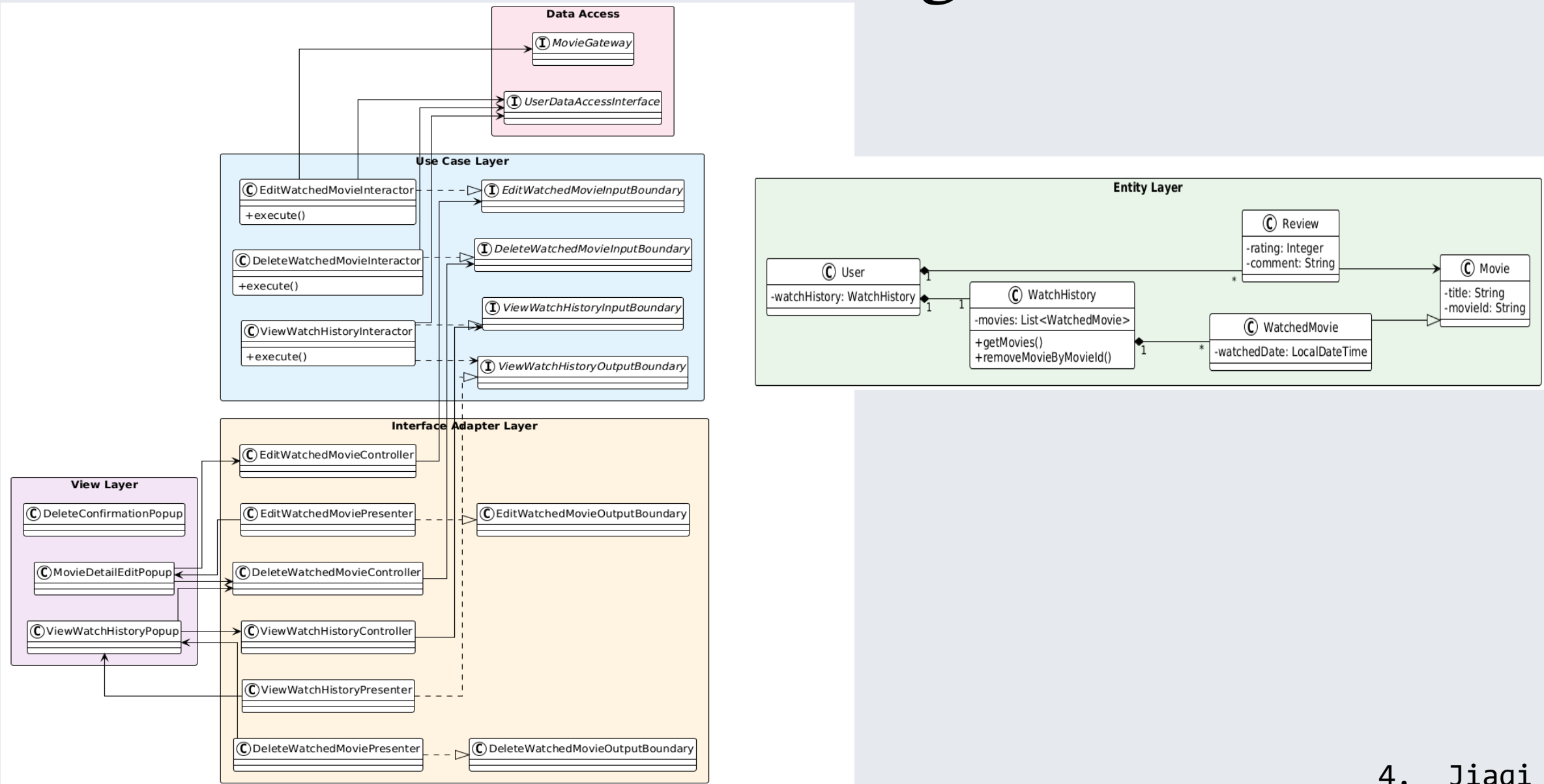
Your Rating (1-5): 4

▼

Your Review:

SaveDeleteBack

- UML Class Diagram -



Use Case Interactor class code

```
import ...

public class ViewWatchHistoryInteractor implements ViewWatchHistoryInputBoundary {

    private final UserDataAccessInterface userDataAccessInterface; 2 usages
    private final ViewWatchHistoryOutputBoundary presenter; 4 usages

    public ViewWatchHistoryInteractor(UserDataAccessInterface userDataAccessInterface,
                                      ViewWatchHistoryOutputBoundary presenter) {
        this.userDataAccessInterface = userDataAccessInterface;
        this.presenter = presenter;
    }
}
```

- **Safely locates the watched movie:** scans the user's [WatchHistory](#) and clearly handles the “movie not found” case.
- **Early failure on invalid state:** calls `prepareFailView` and returns immediately when the watched movie is missing.
- **Validates external movie data:** uses `movieGateway.findById` and stops if the base [Movie](#) cannot be loaded.
- **Protects business rules:** ensures edits only happen when both the history entry and the underlying movie entity are valid.

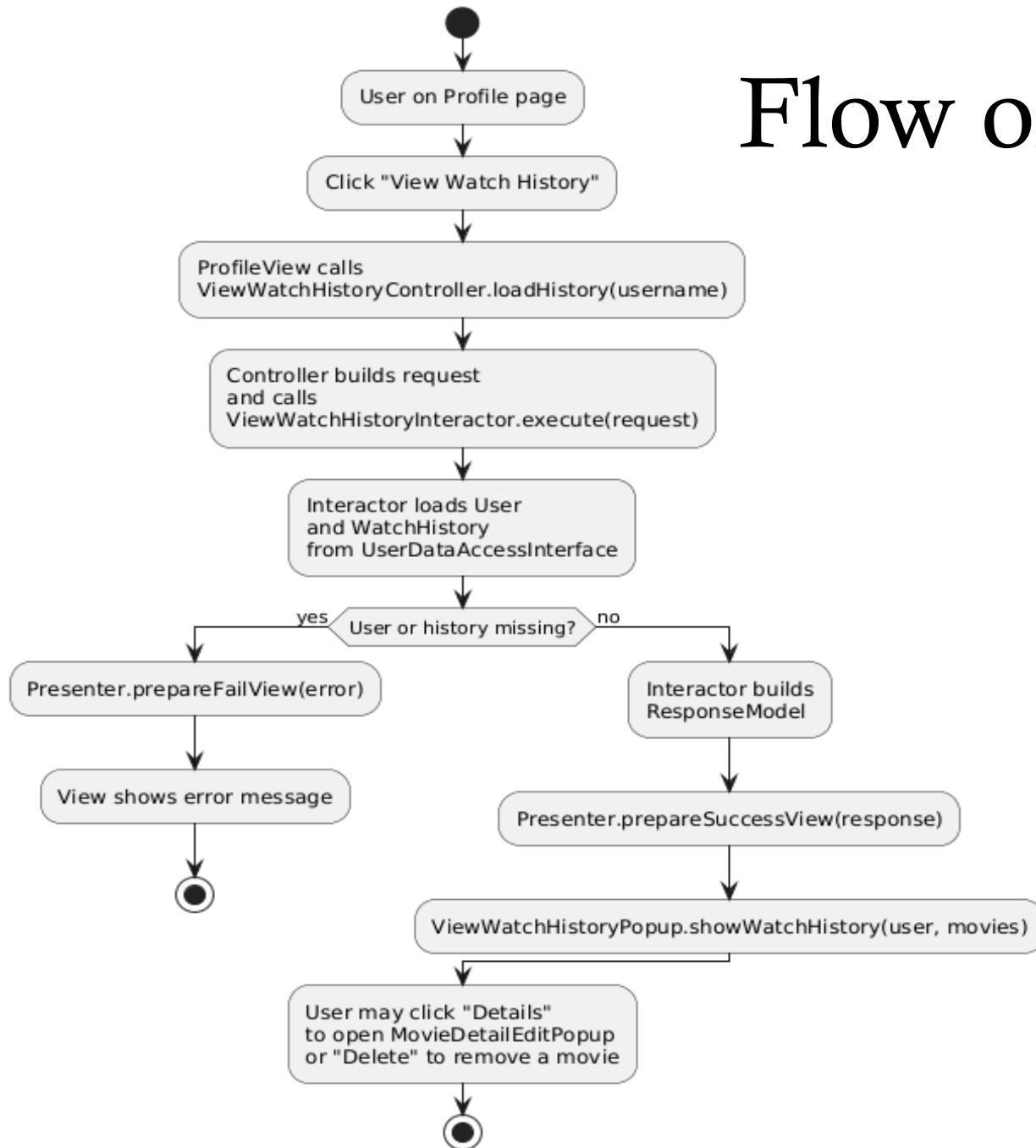
- **Depends only on interfaces:** uses [UserDataAccessInterface](#) and `ViewWatchHistoryOutputBoundary`, not concrete DB or UI classes.
- **Constructor injection:** receives its dependencies via the constructor, which makes the use case easy to test and swap.
- **Clear separation of concerns:** this interactor focuses on loading watch history, leaving presentation and storage to other layers.
- **Fits Clean Architecture:** sits in the use case layer, coordinating between entities and boundaries without knowing about Swing or TMDb.

```
WatchedMovie oldWatchedMovie = null;
for (WatchedMovie wm : user.getWatchHistory().getMovies()) {
    if (requestModel.getMovieId().equals(wm.getMovieId())) {
        oldWatchedMovie = wm;
        break;
    }
}

if (oldWatchedMovie == null) {
    presenter.prepareFailView( errorMessage: "Movie not found in watch history: " + requestModel.getMovieId());
    return;
}

// Get the base Movie entity
Movie movie = movieGateway.findById(requestModel.getMovieId()).orElse( other: null);
if (movie == null) {
    presenter.prepareFailView( errorMessage: "Movie not found: " + requestModel.getMovieId());
    return;
}
```

Flow of Control when Executed

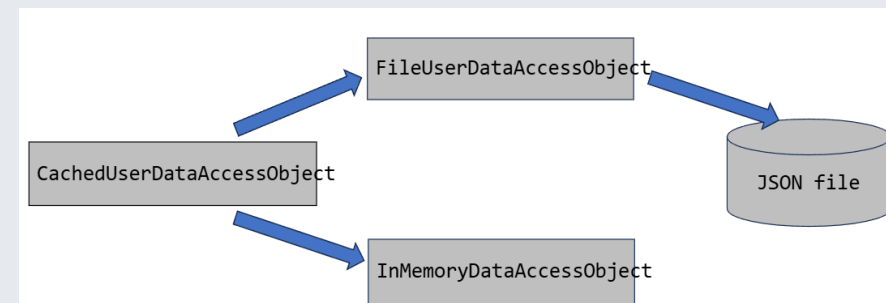


- User clicks “View Watch History” on Profile page
- Controller → calls ViewWatchHistoryInteractor
- Interactor → loads User + WatchHistory from repository
- Presenter → formats response for the view
- ViewWatchHistoryPopup → shows history list + Details / Delete buttons
- Errors / empty history → shown as messages via presenter

Solid Principles

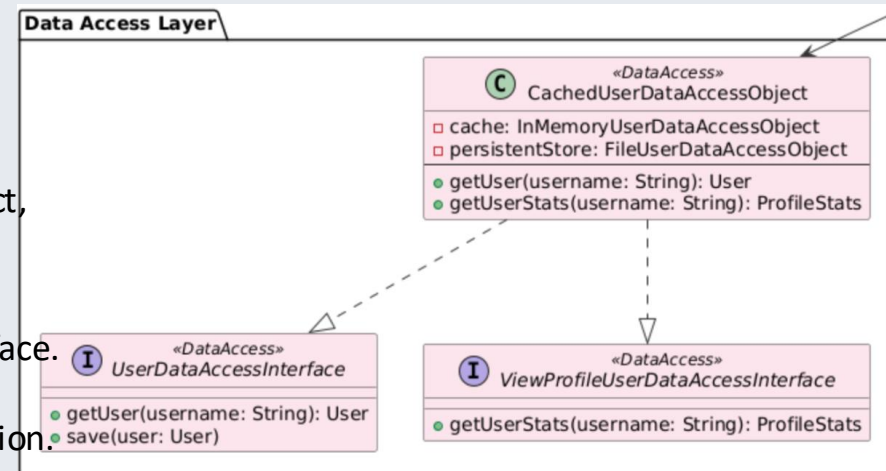
- **Single responsibility principle**

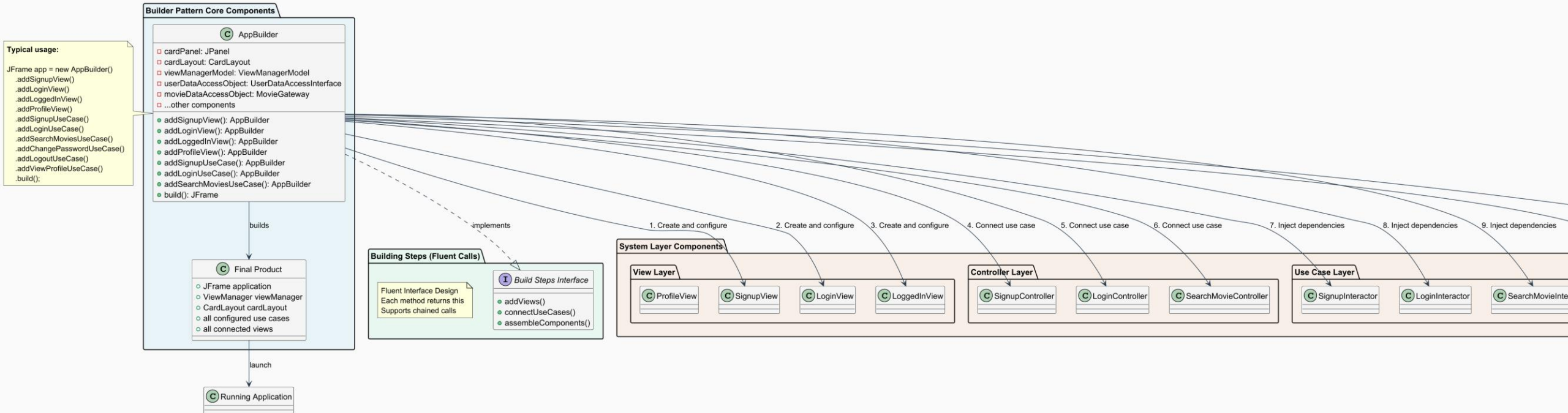
- **FileUserDataAccessObject:**
Its job is single and crystal clear—turn User objects into JSON files and read them back. It doesn't do user validation, it doesn't handle business logic. All it cares about is “object ↔ file” conversion.
- **CachedUserDataAccessObject:**
This one acts as a “cache proxy.” It only goes to the real storage when the cache misses. It follows the single-responsibility principle perfectly.



- **Liskov Substitution principle**

- **UserDataAccessInterface:**
This interface is implemented by InMemoryUserDataAccessObject, FileUserDataAccessObject, and CachedUserDataAccessObject.
- **Future-proofing:**
If we want to add, say, a DatabaseUserDataAccessObject, we just implement the same interface. Everything else in the system keeps working—no changes needed. It's a perfect example of the Open/Closed Principle: open for extension, closed for modification.

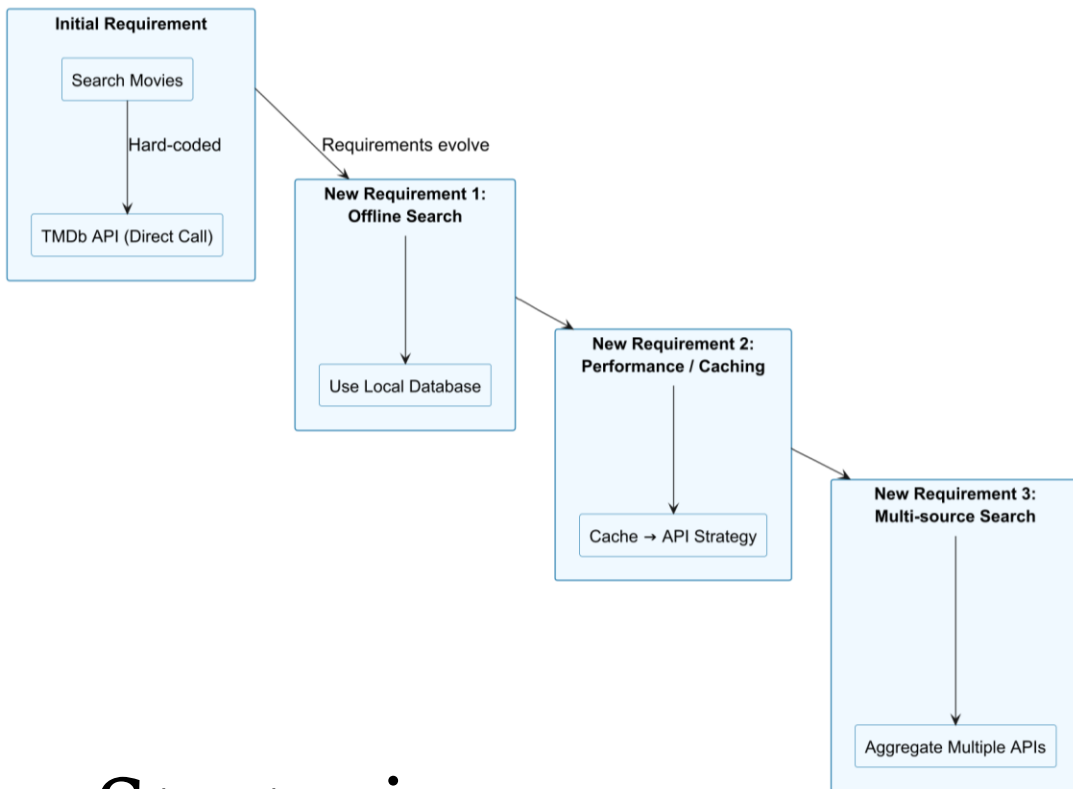




Builder Pattern

- AppBuilder provides a dinner menu of methods like addLoginView() and addSearchMoviesUseCase(). You can pick the features you want, and when you finally call build(), it assembles the entire “application feast” and serves it ready to use.

Strategic Pattern



In our search feature, the MovieGateway interface and its implementations show a “plug-and-play” design.

What problem does it solve? (to maintain Open close prinple)

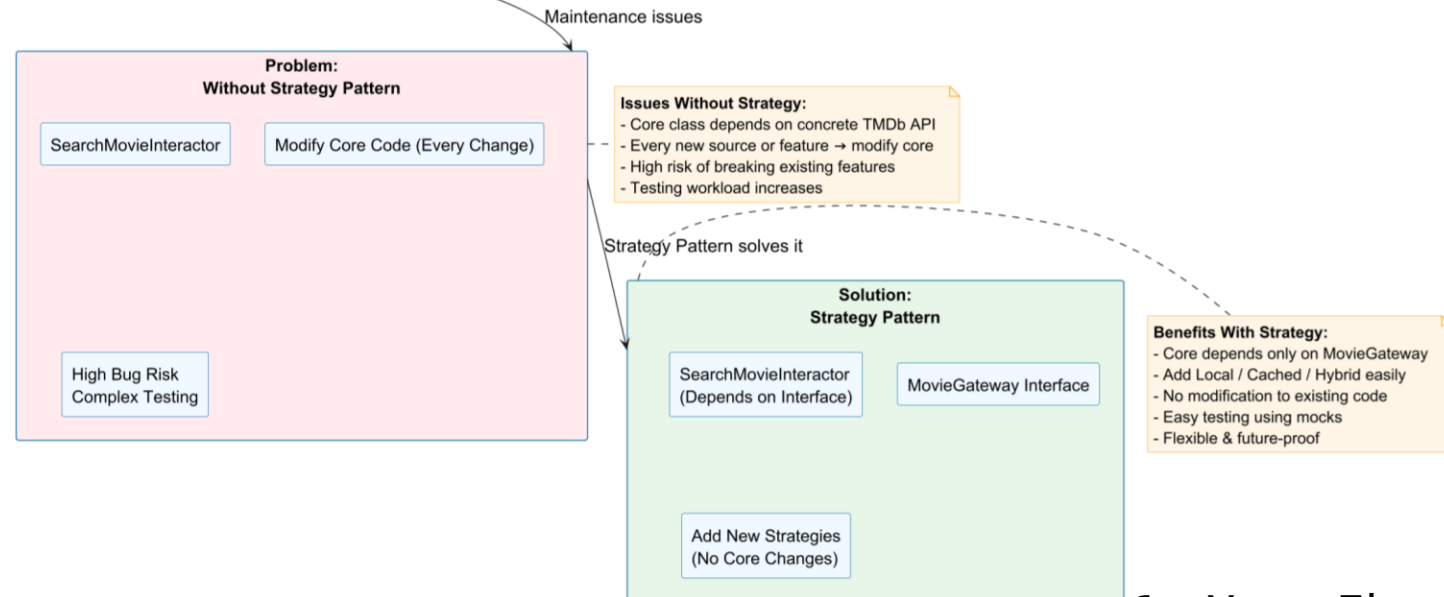
If the SearchMovieInteractor directly called TMDb, adding a local or cached search later would require modifying the core logic.

How does it work?

We define a MovieGateway interface as a “socket” and TMDbMovieDataAccessObject plugs into it as the online search strategy. Future implementations—like LocalMovieGateway or CachedMovieGateway—can be swapped in without touching the core code.

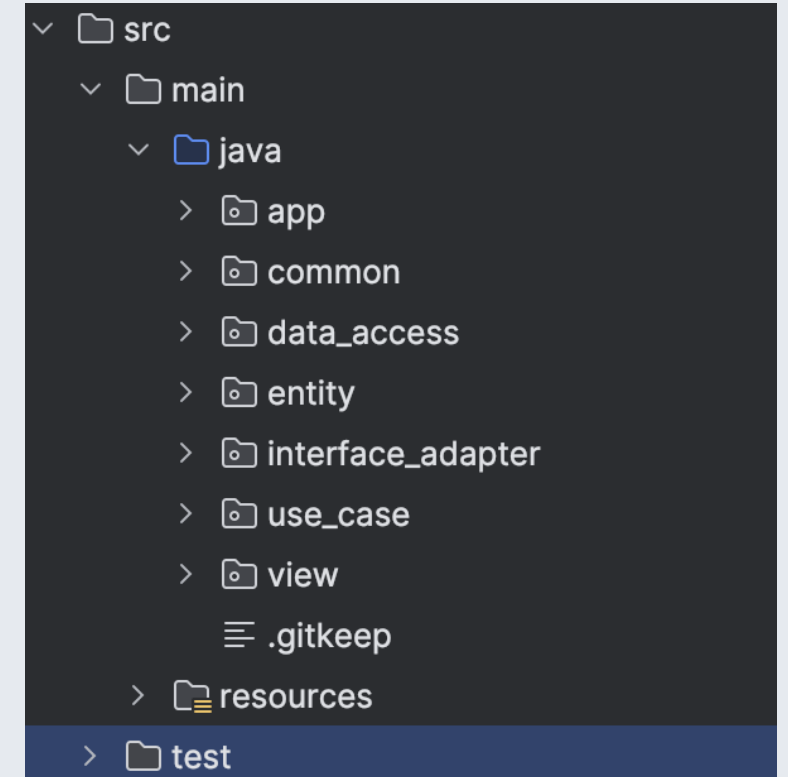
In short:

The Strategy pattern extracts the changeable part (how to search movies) into replaceable plugins, while keeping the core flow unchanged.



Code Organization

- We separated it into packages following the starter code
- Common packages like data access and entity
- entity/ → Business objects (User, Movie, Review)
- use_case/ → Business logic (one folder per use case)
- interface_adapter/ → Controllers & presenters (mirrors use cases)
- view/ → UI components (one Java file per screen)
- data_access/ → External API/database handling
- test/ → separated similarly into data_access, entity, use_case packages



Code Quality

- We maintained code quality by consistently using Checkstyle to enforce formatting, naming conventions, and style rules across the project.
- For example, the pull request on (PR #59) shows how we updated the configuration to better suit our project's needs.
- We also wrote clear and complete Javadoc comments for all classes and methods to ensure readability and maintainability.
- Every feature went through a pull-request workflow, where at least one teammate reviewed the code before merging.

Code Quality

alana-watson commented [3 days ago](#)

Collaborator ...

This PR updates our Checkstyle configurations to better align with standard Java conventions, Clean Architecture practices, and the structure of our program. The primary goal is to reduce false positives and conventions that conflict with our goals, without stepping away from Java and CA best practices.

Key Changes

1. Naming Conventions Updated

Updated `PackageName` regex to allow single layer packages and multi-layer packages with snake case, (e.g. `entity` and `interface_adapter.view_watchlists` are now permitted)

2. `ParameterNumber` modified

Now applies only to methods, not constructors. Prevents false flags on entity constructors like `Movie` (8 params)

3. Lambda and parameter naming conventions fixed

`LambdaParameterName` expanded to allow single letter names (e.g. `e` is very common for `ActionListeners`)

`CatchParameterName` adjusted to allow `e`, `ex`, `err`, etc (was flagging on very standard error calls)

4. Updated `ReturnCount`

Allowed more than 1 return to support readable early-return patterns that are common in controllers and interactors.

5. Removed some misc checks

Removed because they:

- conflict with standard Java idioms
- noisy in the context of a Swing GUI application
- do not meaningfully improve quality in a CA project

Overall should reduce false positives and unnecessary warnings and make style compliance easier!

Questions